

Riyadh PTAP Audit Framework

Complete Repository for: Developing a Climate-Adaptive Audit Framework for Transit Access Points: Assessing Spatial Justice in Riyadh

DIRECTORY STRUCTURE

```
Riyadh-PTAP-Audit-Framework/  
|  
|— README.md  
|— LICENSE  
|— CITATION  
|— .gitignore  
|  
|— config/  
|   |— config.yml  
|  
|— data/  
|   |— raw/  
|       |— metro_stations_2026.csv  
|       |— bus_stops_2025.csv  
|       |— road_network_2024.geojson  
|       |— ward_boundaries_2022.shp  
|       |— ward_boundaries_2022.shx  
|       |— ward_boundaries_2022.dbf  
|       |— ward_boundaries_2022.prj  
|       |   |— population_2022.csv  
|   |— processed/  
|       |— ptap_sample_60.csv  
|       |— field_audit_data.csv  
|       |   |— audit_scores_15_indicators.csv  
|   |— outputs/  
|       |— figures/  
|       |— tables/  
|       |— supplementary/  
|
```

- └─ scripts/
 - └─ 01_data_preparation.R
 - └─ 02_descriptive_statistics.R
 - └─ 03_pca_factor_analysis.R
 - └─ 04_spatial_analysis.R
 - └─ 05_regression_analysis.R
 - └─ 06_latent_class_analysis.R
 - └─ 07_comparative_assessment.R
 - └─ 08_sensitivity_analysis.R
 - └─ 09_figures_tables.R
 - └─ 10_supplementary_analysis.R
 - └─ utils/
 - └─ helpers.R
 - └─ spatial_weights.R
 - └─ statistical_tests.R
- └─ notebooks/
 - └─ 01_exploratory_data_analysis.ipynb
 - └─ 02_supplementary_analysis.ipynb
- └─ results/
 - └─ figures/
 - └─ figure_1_ptap_distribution.png
 - └─ figure_2_conceptual_model.png
 - └─ figure_3_methodological_flowchart.png
 - └─ figure_4_metro_network.png
 - └─ figure_5_bus_density.png
 - └─ figure_6_tucker_congruence.png
 - └─ figure_7_ward_density.png
 - └─ figure_8_lisa_cluster.png
 - └─ tables/
 - └─ table_1_system_characteristics.csv
 - └─ table_2_data_sources.csv
 - └─ table_3_stratification.csv
 - └─ table_4_audit_indicators.csv
 - └─ table_5_descriptive_stats.csv
 - └─ table_6_metro_vs_bus.csv
 - └─ table_7_central_vs_peripheral.csv

```
| | | — table_8_pca_loadings.csv
| | | — table_9_factor_correlations.csv
| | | — table_10_lca_typologies.csv
| | | — table_11_hypothesis_summary.csv
| | | — table_12_comparative_assessment.csv
| | — supplementary/
| |   | — supplementary_figure_S1_photographic_scale.png
| |   | — supplementary_figure_S2_threshold_sensitivity.png
| |   | — supplementary_figure_S3_lca_spatial.png
| |   | — supplementary_table_S1_cross_validation.csv
| |   | — supplementary_table_S2_regression_full.csv
| |   | — supplementary_table_S3_reliability.csv
| |   | — supplementary_table_S4_investment_roadmap.csv
| |   | — supplementary_table_S5_mediation_analysis.csv
| |   | — supplementary_table_S6_threshold_analysis.csv
| |   | — supplementary_table_S9_audit_dataset.csv
| |   | — supplementary_table_S10_getis_ord_results.csv
| |   | — supplementary_table_S11_spatial_weights_sensitivity.csv
| |   | — supplementary_table_S12_correlations.csv
| |   | — supplementary_table_S13_regression_diagnostics.csv
| |   | — supplementary_table_S14_adaptation_framework.csv
| |   | — supplementary_text_S1_path_analysis.md
|
| — dashboard/
|   | — index.html
|   | — style.css
|   | — script.js
|
| — docker/
|   | — Dockerfile
|
| — environment.yml
| — requirements.txt
| — makefile
```

1. README.md

Riyadh PTAP Audit Framework

Climate-Adaptive Audit Framework for Transit Access Points: Assessing Spa

Overview

This repository contains the complete reproducible research materials for:

****"Developing a Climate-Adaptive Audit Framework for Transit Access Points: A**

****Author:**** Majed M Halawani

****Affiliation:**** Urban Planning Department, College of Architecture and Planning,

****Email:**** nandigk0@gmail.com

****Journal:**** Journal of Advanced Transportation

****Manuscript ID:**** 9356236

Abstract

This study develops and internally tests a 15-indicator climate-adaptive screening

Repository Contents

Directory	Description
`config/`	Configuration files for analysis parameters
`data/`	Raw, processed, and output data files
`scripts/`	All R analysis scripts (numbered sequentially)
`notebooks/`	Jupyter notebooks for exploratory analysis
`results/`	Generated figures, tables, and supplementary materials
`dashboard/`	Interactive web dashboard
`docker/`	Docker configuration for reproducibility

Key Findings

- ****Overall infrastructure quality:**** Mean composite score = 0.61 (SD = 0.18)
- ****Metro vs. Bus:**** Metro stations significantly outperform bus stops (Cohen's d

- **Central vs. Peripheral:** Central wards outperform peripheral wards (Cohen's $d = 0.45$)
- **Spatial clustering:** Global Moran's $I = 0.52$ ($p < 0.001$)
- **Five-factor structure:** Quality of Access, Footpath Space, Crossing Time, Ease of Access, and Safety
- **PTAP typologies:** High-Quality Integrated (30%), Moderate with Lighting Design (40%), and Basic (30%)

Data Sources

Dataset	Source	Year	Format	Resolution
Metro stations	RCRC Open Data Portal	2026	GeoJSON, CSV	94 points
Bus stops	RCRC Open Data Portal	2025	CSV, GeoJSON	3,010 points
Road network	Overture Maps	2024	PMTiles, GeoParquet	423,735 links
Ward boundaries	GASTAT	2022	Shapefile	72 wards
Population	GASTAT	2022	CSV	Ward level
Field audit data	Primary data collection	2026	CSV	60 PTAPs

Software Requirements

R Environment

- R version: 4.3.2 or higher
- Key packages: tidyverse, sf, psych, lavaan, poLCA, spdep, ggplot2, corrplot, haven

Python Environment

- Python version: 3.10 or higher
- Key packages: pandas, numpy, geopandas, matplotlib, seaborn, scikit-learn, scipy

Installation

Option 1: Using Conda (Recommended)

```

```bash
conda env create -f environment.yml
conda activate riyadh-ptap

```

Option 2: Using pip

```

pip install -r requirements.txt

```

### Option 3: Using Docker

```
docker build -t riyadh-ptap -f docker/Dockerfile .
docker run -it -v $(pwd):/workspace riyadh-ptap
```

## Reproducibility Instructions

### Step 1: Prepare Data

Place all raw data files in data/raw/ directory.

### Step 2: Run All Analyses

Execute R scripts in numerical order:

```
Rscript scripts/01_data_preparation.R
Rscript scripts/02_descriptive_statistics.R
Rscript scripts/03_pca_factor_analysis.R
Rscript scripts/04_spatial_analysis.R
Rscript scripts/05_regression_analysis.R
Rscript scripts/06_latent_class_analysis.R
Rscript scripts/07_comparative_assessment.R
Rscript scripts/08_sensitivity_analysis.R
Rscript scripts/09_figures_tables.R
Rscript scripts/10_supplementary_analysis.R
```

### Step 3: Generate All Outputs

All figures, tables, and supplementary materials will be generated in results/ directory.

## Interactive Dashboard

Open dashboard/index.html in any modern web browser to explore:

- Infrastructure quality scores by PTAP
- Spatial clustering patterns (LISA)
- LCA typology classifications
- Indicator-level deficiencies
- Interactive maps

## Key Statistical Tests

Test	Description	Result
Tucker's congruence	Factor structure stability	$\phi = 0.91$
Cronbach's alpha	Internal consistency	$\alpha = 0.82$
Cohen's kappa	Inter-rater reliability	$\kappa = 0.84$
KMO	Sampling adequacy	0.79
Bartlett's test	Sphericity	$\chi^2 = 412.6, p < 0.001$
Global Moran's I	Spatial autocorrelation	$I = 0.52, p < 0.001$
MANOVA	Metro vs. Bus differences	Wilks' $\lambda = 0.58, p < 0.001$

## Output Files

All results are stored in results/:

### Figures (8 main + 3 supplementary):

- Figure 1: PTAP spatial distribution
- Figure 2: Conceptual model
- Figure 3: Methodological flowchart
- Figure 4: Metro network
- Figure 5: Bus stop density
- Figure 6: Tucker's congruence plot
- Figure 7: Ward-level density comparison
- Figure 8: LISA cluster map

### Tables (12 main + 14 supplementary):

- Table 1-12: All main results tables
- Table S1-S14: All supplementary tables

## Citation

```
@article{halawani2026climate,
 title={Developing a Climate-Adaptive Audit Framework for Transit Access Points:},
 author={Halawani, Majed M},
 journal={Journal of Advanced Transportation},
 year={2026}
}
```

## License

MIT License - see LICENSE file for details.

## Contact

Majed M Halawani

Email: [nandigk0@gmail.com](mailto:nandigk0@gmail.com)

Urban Planning Department, College of Architecture and Planning, King Saud University, Riyadh, Saudi Arabia

## Acknowledgments

This research was supported by the Ongoing Research Funding program (ORF-2026-1901), King Saud University, Riyadh, Saudi Arabia.

---

## 2. LICENSE

``text

MIT License

Copyright (c) 2026 Majed M Halawani

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

---

## 3. CITATION

Halawani, M.M. (2026). Developing a Climate-Adaptive Audit Framework for Transit

@article{halawani2026climate,

title={Developing a Climate-Adaptive Audit Framework for Transit Access Points:

```
author={Halawani, Majed M},
journal={Journal of Advanced Transportation},
year={2026},
note={Manuscript ID: 9356236}
}
```

---

#### 4. .gitignore

```
=====
R
=====
.Rproj.user
.Rhistory
.RData
.Ruserdata
.Rproj
.Rapp.history
.Renvirom
.Rbuildignore
*.Rproj

=====
Python
=====
__pycache__/
*.pyc
*.pyo
*.pyd
*.so
*.egg
*.egg-info/
dist/
build/
.Python
env/
venv/
.venv/
```

```

pip-log.txt
pip-delete-this-directory.txt

=====
Jupyter Notebooks
=====
.ipynb_checkpoints/
*.ipynb_checkpoints

=====
Data Files
=====
data/raw/*
!data/raw/.gitkeep
data/processed/*
!data/processed/.gitkeep
results/figures/*.png
!results/figures/.gitkeep
results/figures/*.pdf
!results/figures/.gitkeep
results/tables/*.csv
!results/tables/.gitkeep
results/supplementary/*.csv
!results/supplementary/.gitkeep
results/supplementary/*.png
!results/supplementary/.gitkeep

=====
IDE and OS
=====
.vscode/
.idea/
*.swp
*.swo
*~
.DS_Store
Thumbs.db
Desktop.ini

```

```
=====
Logs and Temporary Files
=====
*.log
*.tmp
*.temp
*.cache
```

---

## 5. config/config.yml

```
=====
Riyadh PTAP Audit Framework - Configuration File
=====
Author: Majed M Halawani
Date: 2026
Description: Master configuration for all analyses
=====

project:
 name: "Riyadh-PTAP-Audit-Framework"
 version: "1.0.0"
 author: "Majed M Halawani"
 affiliation: "King Saud University"
 year: 2026
 manuscript_id: "9356236"
 journal: "Journal of Advanced Transportation"

paths:
 raw_data: "data/raw/"
 processed_data: "data/processed/"
 outputs: "results/"
 figures: "results/figures/"
 tables: "results/tables/"
 supplementary: "results/supplementary/"
 dashboard: "dashboard/"
 notebooks: "notebooks/"
```

```
scripts: "scripts/"
utils: "scripts/utils/"
```

sampling:

```
seed: 2026
metro_stations_total: 30
bus_stops_total: 30
total_ptaps: 60
passenger_density:
```

strata:

- "high"
- "medium"
- "low"

thresholds:

```
high: 50000
medium_low: 20000
medium_high: 50000
low: 20000
```

land\_use:

types:

- "residential"
- "commercial"
- "mixed"
- "institutional"

counts:

```
residential: 16
commercial: 16
mixed: 14
institutional: 14
```

spatial:

```
buffer_distance: 500 # meters
kernel_bandwidth: 500 # meters
central_threshold_km: 5 # kilometers
population_density_threshold: 5000 # persons per km2
inverse_distance_threshold_km: 5
getis_ord_windows:
- 1
```

- 2
- 3

statistics:

alpha: 0.05

bootstrap\_iterations: 1000

pca:

rotation: "oblimin"

eigenvalue\_threshold: 1.0

factor\_retention\_criterion: "parallel\_analysis"

n\_factors: 5

lca:

max\_classes: 5

nstart: 100

criterion: "BIC"

regression:

vif\_threshold: 5

heteroscedasticity\_test: "breusch\_pagan"

normality\_test: "shapiro\_wilk"

indicators:

total\_indicators: 15

names:

- "pavement\_type"
- "sidewalk\_width"
- "curb\_height"
- "cleanliness"
- "obstructions"
- "buffer\_zone"
- "pedestrian\_signage"
- "crossing\_frequency"
- "crossing\_type"
- "crossing\_time"
- "disability\_infrastructure"
- "night\_lighting"
- "adjacent\_land\_use"
- "amenities"
- "access\_time"

safety\_indicators:

- "buffer\_zone"
- "crossing\_frequency"
- "crossing\_type"
- "crossing\_time"
- "night\_lighting"

weights:

safety: 2.0

standard: 1.0

figure\_formats:

- "png"
- "pdf"
- "svg"

reliability:

inter\_rater\_percentage: 0.20 *# 20% of sample*

cohen\_kappa\_threshold: 0.70

cronbach\_alpha\_threshold: 0.70

analysis:

hypotheses:

H1: "metro\_bus\_difference"

H2: "central\_peripheral\_difference"

H3: "spatial\_clustering"

H4: "interaction\_type\_location"

propositions:

P1: "mediation\_analysis"

P2: "three\_way\_interaction"

P3: "threshold\_analysis"

P4: "behavioural\_adaptation"

---

## 6. scripts/01\_data\_preparation.R

```
#!/usr/bin/env Rscript
```

```
=====
```

```
01_data_preparation.R
```

```

Data Preparation Script
Riyadh PTAP Audit Framework
=====
Author: Majed M Halawani
Description: Loads raw data, performs cleaning, and creates processed datasets
=====

1. Environment Setup

Load required libraries
suppressPackageStartupMessages({
 library(tidyverse)
 library(sf)
 library(here)
 library(yaml)
 library(janitor)
 library(lubridate)
})

Load configuration
config <- yaml::read_yaml(here("config/config.yml"))

Set seed for reproducibility
set.seed(config$sampling$seed)

Define paths
raw_path <- here(config$paths$raw_data)
processed_path <- here(config$paths$processed_data)

Create directories if they don't exist
dir.create(raw_path, recursive = TRUE, showWarnings = FALSE)
dir.create(processed_path, recursive = TRUE, showWarnings = FALSE)

2. Logging Function

```

```

log_message <- function(msg, level = "INFO") {
 timestamp <- format(Sys.time(), "%Y-%m-%d %H:%M:%S")
 cat(sprintf("[%s] %s: %s\n", timestamp, level, msg))
}

log_message("Starting data preparation")

3. Load Raw Data

log_message("Loading raw data files...")

3.1 Metro Stations
metro_file <- file.path(raw_path, "metro_stations_2026.csv")
if (file.exists(metro_file)) {
 metro_raw <- read_csv(metro_file, show_col_types = FALSE)
 log_message(sprintf(" Metro stations loaded: %d", nrow(metro_raw)))
} else {
 log_message(" WARNING: metro_stations_2026.csv not found", "WARN")
 metro_raw <- data.frame()
}

3.2 Bus Stops
bus_file <- file.path(raw_path, "bus_stops_2025.csv")
if (file.exists(bus_file)) {
 bus_raw <- read_csv(bus_file, show_col_types = FALSE)
 log_message(sprintf(" Bus stops loaded: %d", nrow(bus_raw)))
} else {
 log_message(" WARNING: bus_stops_2025.csv not found", "WARN")
 bus_raw <- data.frame()
}

3.3 Ward Boundaries (Shapefile)
ward_dir <- raw_path
ward_files <- list.files(ward_dir, pattern = "ward_boundaries.*\\.shp$", full.name:
if (length(ward_files) > 0) {

```

```

ward_raw <- st_read(ward_files[1], quiet = TRUE)
log_message(sprintf(" Ward boundaries loaded: %d polygons", nrow(ward_raw))
} else {
 log_message(" WARNING: ward_boundaries shapefile not found", "WARN")
 ward_raw <- st_sf()
}

```

### *# 3.4 Population Data*

```

pop_file <- file.path(raw_path, "population_2022.csv")
if (file.exists(pop_file)) {
 pop_raw <- read_csv(pop_file, show_col_types = FALSE)
 log_message(sprintf(" Population data loaded: %d records", nrow(pop_raw)))
} else {
 log_message(" WARNING: population_2022.csv not found", "WARN")
 pop_raw <- data.frame()
}

```

### *# 3.5 Road Network*

```

road_file <- file.path(raw_path, "road_network_2024.geojson")
if (file.exists(road_file)) {
 road_raw <- st_read(road_file, quiet = TRUE)
 log_message(sprintf(" Road network loaded: %d links", nrow(road_raw)))
} else {
 log_message(" WARNING: road_network_2024.geojson not found", "WARN")
 road_raw <- st_sf()
}

```

```

4. Data Cleaning and Standardization

```

```

log_message("Cleaning and standardizing data...")

```

### *# 4.1 Metro Stations*

```

if (nrow(metro_raw) > 0) {
 metro_clean <- metro_raw %>%
 clean_names() %>%
 mutate(

```

```

station_id = paste0("M", sprintf("%03d", row_number())),
type = "metro",
source = "RCRC_2026",
Standardize coordinate columns
longitude = as.numeric(coalesce(longitude, lon, x, long)),
latitude = as.numeric(coalesce(latitude, lat, y, latit)),
Clean line information
line = as.factor(coalesce(line, metro_line, route, line_name)),
station_name = coalesce(name, station_name, stop_name, "Unknown"),
status = coalesce(status, "operational")
) %>%
filter(!is.na(longitude) & !is.na(latitude)) %>%
select(station_id, type, station_name, line, longitude, latitude,
 status, everything(), -matches("geometry"))

log_message(sprintf(" Metro stations cleaned: %d", nrow(metro_clean)))
} else {
 metro_clean <- data.frame()
}

4.2 Bus Stops
if (nrow(bus_raw) > 0) {
 bus_clean <- bus_raw %>%
 clean_names() %>%
 mutate(
 stop_id = paste0("B", sprintf("%05d", row_number())),
 type = "bus",
 source = "RCRC_2025",
 # Standardize coordinate columns
 longitude = as.numeric(coalesce(longitude, lon, x, long)),
 latitude = as.numeric(coalesce(latitude, lat, y, latit)),
 # Route count
 route_count = as.numeric(coalesce(route_count, routes, n_routes, 0)),
 stop_name = coalesce(name, stop_name, location, "Unknown")
) %>%
 filter(!is.na(longitude) & !is.na(latitude)) %>%
 select(stop_id, type, stop_name, route_count, longitude, latitude,
 everything(), -matches("geometry"))

```

```

log_message(sprintf(" Bus stops cleaned: %d", nrow(bus_clean)))
} else {
 bus_clean <- data.frame()
}

4.3 Ward Boundaries
if (nrow(ward_raw) > 0) {
 ward_clean <- ward_raw %>%
 st_make_valid() %>%
 mutate(
 ward_id = row_number(),
 ward_name = coalesce(ward_name, name, district, "Unknown")
)

 # Ensure coordinate reference system is set
 if (is.na(st_crs(ward_clean))) {
 st_crs(ward_clean) <- 4326
 }

 # Project to metric CRS for area calculations
 ward_clean_metric <- st_transform(ward_clean, 3857)
 ward_clean$area_km2 <- as.numeric(st_area(ward_clean_metric)) / 1e6

 log_message(sprintf(" Wards cleaned: %d polygons", nrow(ward_clean)))
} else {
 ward_clean <- st_sf()
}

4.4 Population Data
if (nrow(pop_raw) > 0) {
 pop_clean <- pop_raw %>%
 clean_names() %>%
 mutate(
 ward_id = as.numeric(coalesce(ward_id, district_id, id, row_number())),
 population = as.numeric(coalesce(population, pop, total_pop, 0)),
 household_count = as.numeric(coalesce(households, household_count, 0)),
 ward_name = coalesce(ward_name, district, name, "Unknown")
)
}

```

```

) %>%
 select(ward_id, ward_name, population, household_count, everything())

log_message(sprintf(" Population data cleaned: %d records", nrow(pop_clean)))
} else {
 pop_clean <- data.frame()
}

4.5 Combine Population with Ward Boundaries
if (nrow(ward_clean) > 0 && nrow(pop_clean) > 0) {
 ward_with_pop <- ward_clean %>%
 left_join(pop_clean, by = "ward_id")

 # Calculate population density
 ward_with_pop <- ward_with_pop %>%
 mutate(
 population_density = population / area_km2,
 population_density = ifelse(is.na(population_density) | is.infinite(population_d
 0, population_density)
)

 log_message(" Ward data merged with population")
} else {
 ward_with_pop <- ward_clean
}

5. Combine PTAPs

log_message("Combining all PTAPs...")

all_ptaps <- data.frame()

5.1 Metro stations
if (nrow(metro_clean) > 0) {
 metro_ptaps <- metro_clean %>%
 select(

```

```

 ptap_id = station_id,
 type,
 name = station_name,
 line,
 longitude,
 latitude,
 status
) %>%
 mutate(
 route_count = NA,
 density_stratum = NA
)
all_ptaps <- bind_rows(all_ptaps, metro_ptaps)
}

```

*# 5.2 Bus stops*

```

if (nrow(bus_clean) > 0) {
 bus_ptaps <- bus_clean %>%
 select(
 ptap_id = stop_id,
 type,
 name = stop_name,
 route_count,
 longitude,
 latitude
) %>%
 mutate(
 line = NA,
 status = "operational",
 density_stratum = NA
)
 all_ptaps <- bind_rows(all_ptaps, bus_ptaps)
}

```

```

log_message(sprintf(" Total PTAPs: %d", nrow(all_ptaps)))
log_message(sprintf(" Metro: %d", sum(all_ptaps$type == "metro")))
log_message(sprintf(" Bus: %d", sum(all_ptaps$type == "bus")))

```

```

5.3 Add PTAP density by ward
if (nrow(ward_with_pop) > 0) {
 all_ptaps_sf <- st_as_sf(all_ptaps, coords = c("longitude", "latitude"), crs = 4326)
 all_ptaps_sf <- st_join(all_ptaps_sf, ward_with_pop["ward_id"])

 all_ptaps <- all_ptaps_sf %>%
 st_drop_geometry() %>%
 mutate(ward_id = as.numeric(ward_id))

 log_message(" PTAPs assigned to wards")
}

6. Save Cleaned Data

log_message("Saving cleaned data...")

6.1 Save processed datasets
write_csv(metro_clean, file.path(processed_path, "metro_stations_clean.csv"))
write_csv(bus_clean, file.path(processed_path, "bus_stops_clean.csv"))
write_csv(all_ptaps, file.path(processed_path, "all_ptaps_clean.csv"))
write_csv(pop_clean, file.path(processed_path, "population_clean.csv"))

6.2 Save ward boundaries (as shapefile and GeoJSON)
if (nrow(ward_with_pop) > 0) {
 st_write(ward_with_pop, file.path(processed_path, "ward_boundaries_clean.shp"),
 delete_layer = TRUE, quiet = TRUE)
 st_write(ward_with_pop, file.path(processed_path, "ward_boundaries_clean.geojson"),
 delete_layer = TRUE, quiet = TRUE)
}

6.3 Save road network (if available)
if (nrow(road_raw) > 0) {
 st_write(road_raw, file.path(processed_path, "road_network_clean.geojson"),
 delete_layer = TRUE, quiet = TRUE)
}

```

```

6.4 Save configuration summary
config_summary <- tibble(
 parameter = c(
 "total_metro_stations",
 "total_bus_stops",
 "total_ptaps",
 "seed",
 "central_threshold_km"
),
 value = c(
 nrow(metro_clean),
 nrow(bus_clean),
 nrow(all_ptaps),
 config$sampling$seed,
 config$spatial$central_threshold_km
)
)
write_csv(config_summary, file.path(processed_path, "config_summary.csv"))

7. Generate Sample Metadata

log_message("Generating sample metadata...")

sample_metadata <- all_ptaps %>%
 group_by(type) %>%
 summarise(
 count = n(),
 has_longitude = sum(!is.na(longitude)),
 has_latitude = sum(!is.na(latitude))
)

write_csv(sample_metadata, file.path(processed_path, "sample_metadata.csv"))

log_message("Data preparation complete!")
log_message(" Outputs saved to:", processed_path)

```

```

8. Cleanup

Print session info
sessionInfo()
```

---

## 7. scripts/02\_descriptive\_statistics.R

```
#!/usr/bin/env Rscript
=====
02_descriptive_statistics.R
Descriptive Statistics Script
Riyadh PTAP Audit Framework
=====
Author: Majed M Halawani
Description: Calculates descriptive statistics for all indicators
=====

1. Environment Setup

suppressPackageStartupMessages({
 library(tidyverse)
 library(here)
 library(yaml)
 library(psych)
 library(corrplot)
})

Load configuration
config <- yaml::read_yaml(here("config/config.yml"))
set.seed(config$sampling$seed)

Define paths
processed_path <- here(config$paths$processed_data)
```

```

output_path <- here(config$paths$outputs)

Create directories
dir.create(file.path(output_path, "tables"), recursive = TRUE, showWarnings = FALSE)
dir.create(file.path(output_path, "figures"), recursive = TRUE, showWarnings = FALSE)

2. Load Data

cat("Loading data for descriptive statistics...\n")

Load cleaned data
all_ptaps <- read_csv(file.path(processed_path, "all_ptaps_clean.csv"), show_col_types = FALSE)

Load field audit data (should contain the 15 indicators)
audit_file <- file.path(processed_path, "field_audit_data.csv")
if (file.exists(audit_file)) {
 audit_data <- read_csv(audit_file, show_col_types = FALSE)
} else {
 # Create synthetic audit data for demonstration
 cat(" NOTE: field_audit_data.csv not found. Using synthetic data for testing.\n")
 # In practice, this would be replaced with actual audit data
 n_ptaps <- 60
 audit_data <- tibble(
 ptap_id = all_ptaps$ptap_id[1:n_ptaps],
 type = all_ptaps$type[1:n_ptaps],
 pavement_type = round(runif(n_ptaps, 0.2, 1.0), 2),
 sidewalk_width = round(runif(n_ptaps, 0.2, 1.0), 2),
 curb_height = round(runif(n_ptaps, 0.2, 1.0), 2),
 cleanliness = round(runif(n_ptaps, 0.2, 1.0), 2),
 obstructions = round(runif(n_ptaps, 0.2, 1.0), 2),
 buffer_zone = round(runif(n_ptaps, 0.2, 1.0), 2),
 pedestrian_signage = round(runif(n_ptaps, 0.2, 1.0), 2),
 crossing_frequency = round(runif(n_ptaps, 0.2, 1.0), 2),
 crossing_type = round(runif(n_ptaps, 0.2, 1.0), 2),
 crossing_time = round(runif(n_ptaps, 0.2, 1.0), 2),
 disability_infrastructure = round(runif(n_ptaps, 0.2, 1.0), 2),
)
}

```

```

 night_lighting = round(runif(n_ptaps, 0.2, 1.0), 2),
 adjacent_land_use = round(runif(n_ptaps, 0.2, 1.0), 2),
 amenities = round(runif(n_ptaps, 0.2, 1.0), 2),
 access_time = round(runif(n_ptaps, 0.2, 1.0), 2)
)
}

cat(sprintf(" Loaded audit data for %d PTAPs\n", nrow(audit_data)))

3. Define Indicator Names

indicator_names <- config$indicators$names
safety_indicators <- config$indicators$safety_indicators

cat("\nIndicators:\n")
cat(sprintf(" Total: %d\n", length(indicator_names)))
cat(sprintf(" Safety indicators: %s\n", paste(safety_indicators, collapse = ", ")))

4. Calculate Composite Scores

cat("\nCalculating composite scores...\n")

4.1 Function to calculate composite score
calculate_composite <- function(data, indicator_cols, safety_cols, safety_weight
 # Standardize each indicator (0-1 scale)
 standardized <- data %>%
 select(all_of(indicator_cols)) %>%
 mutate(across(everything(), ~ . / max(., na.rm = TRUE)))

 # Apply weights
 weighted_scores <- standardized %>%
 mutate(across(all_of(safety_cols), ~ . * safety_weight))

 # Calculate composite (mean of weighted scores)

```

```

composite <- rowMeans(weighted_scores, na.rm = TRUE)

return(composite)
}

Get indicator columns
indicator_cols <- intersect(indicator_names, names(audit_data))
safety_cols <- intersect(safety_indicators, names(audit_data))

cat(sprintf(" Found %d indicators in data\n", length(indicator_cols)))
cat(sprintf(" Found %d safety indicators\n", length(safety_cols)))

4.2 Calculate composite scores
audit_data <- audit_data %>%
 mutate(
 composite_score = calculate_composite(
 .,
 indicator_cols = indicator_cols,
 safety_cols = safety_cols,
 safety_weight = config$indicators$weights$safety
)
)

4.3 Calculate individual scores with equal weights (for sensitivity)
audit_data <- audit_data %>%
 mutate(
 composite_score_equal = calculate_composite(
 .,
 indicator_cols = indicator_cols,
 safety_cols = safety_cols,
 safety_weight = 1.0
)
)

cat(" Composite scores calculated\n")

5. Descriptive Statistics

```

```

cat("\nCalculating descriptive statistics...\n")

5.1 Overall statistics
overall_stats <- audit_data %>%
 summarise(
 n = n(),
 mean_score = mean(composite_score, na.rm = TRUE),
 sd_score = sd(composite_score, na.rm = TRUE),
 min_score = min(composite_score, na.rm = TRUE),
 max_score = max(composite_score, na.rm = TRUE),
 q25 = quantile(composite_score, 0.25, na.rm = TRUE),
 median_score = median(composite_score, na.rm = TRUE),
 q75 = quantile(composite_score, 0.75, na.rm = TRUE),
 iqr = q75 - q25
)

cat(sprintf(" Overall: mean = %.3f, SD = %.3f\n",
 overall_stats$mean_score, overall_stats$sd_score))

5.2 Statistics by PTAP type
type_stats <- audit_data %>%
 group_by(type) %>%
 summarise(
 n = n(),
 mean_score = mean(composite_score, na.rm = TRUE),
 sd_score = sd(composite_score, na.rm = TRUE),
 min_score = min(composite_score, na.rm = TRUE),
 max_score = max(composite_score, na.rm = TRUE)
)

cat(" By type:\n")
for (i in 1:nrow(type_stats)) {
 cat(sprintf(" %s: mean = %.3f, SD = %.3f\n",
 type_stats$type[i], type_stats$mean_score[i], type_stats$sd_score[i]))
}

```

### *# 5.3 Statistics by indicator*

```
indicator_stats <- audit_data %>%
 select(all_of(indicator_cols)) %>%
 pivot_longer(everything(), names_to = "indicator", values_to = "score") %>%
 group_by(indicator) %>%
 summarise(
 mean_score = mean(score, na.rm = TRUE),
 sd_score = sd(score, na.rm = TRUE),
 min_score = min(score, na.rm = TRUE),
 max_score = max(score, na.rm = TRUE)
) %>%
 arrange(desc(mean_score))

cat(" Top 5 indicators by mean score:\n")
for (i in 1:min(5, nrow(indicator_stats))) {
 cat(sprintf(" %s: %.3f\n", indicator_stats$indicator[i], indicator_stats$mean_score[i]))
}
```

```

6. Reliability Analysis

```

```
cat("\nCalculating reliability...\n")
```

#### *# 6.1 Cronbach's alpha*

```
if (length(indicator_cols) >= 2) {
 alpha_result <- psych::alpha(audit_data %>% select(all_of(indicator_cols)),
 check.keys = TRUE)
 cronbach_alpha <- alpha_result$total$raw_alpha
 cat(sprintf(" Cronbach's alpha: %.3f\n", cronbach_alpha))
} else {
 cronbach_alpha <- NA
 cat(" Warning: Insufficient indicators for alpha calculation\n")
}
```

#### *# 6.2 Inter-item correlations*

```
cor_matrix <- audit_data %>%
 select(all_of(indicator_cols)) %>%
```

```

cor(use = "pairwise.complete.obs")

7. Save Results

cat("\nSaving results...\n")

7.1 Save descriptive statistics
write_csv(overall_stats, file.path(output_path, "tables/descriptive_overall.csv"))
write_csv(type_stats, file.path(output_path, "tables/descriptive_by_type.csv"))
write_csv(indicator_stats, file.path(output_path, "tables/descriptive_by_indicat

7.2 Save correlation matrix
write_csv(as.data.frame(cor_matrix), file.path(output_path, "tables/correlation_

7.3 Save reliability results
reliability_summary <- tibble(
 metric = c("cronbach_alpha"),
 value = cronbach_alpha
)
write_csv(reliability_summary, file.path(output_path, "tables/reliability_summar

7.4 Save audited data with composite scores
write_csv(audit_data, file.path(processed_path, "audit_data_with_scores.csv"))

7.5 Generate correlation plot
png(file.path(output_path, "figures/correlation_matrix.png"),
 width = 10, height = 10, units = "in", res = 300)
corrplot::corrplot(cor_matrix,
 method = "color",
 type = "upper",
 tl.col = "black",
 tl.srt = 45,
 addCoef.col = "black",
 number.cex = 0.6)
dev.off()
cat(" Correlation matrix plot saved\n")

```

```

8. Summary Report

cat("\n" + paste(rep("=", 60), collapse = "") + "\n")
cat("DESCRIPTIVE STATISTICS SUMMARY REPORT\n")
cat(paste(rep("=", 60), collapse = "") + "\n\n")

cat(sprintf("Total PTAPs audited: %d\n", overall_stats$n))
cat(sprintf("Overall mean composite score: %.3f (SD = %.3f)\n",
 overall_stats$mean_score, overall_stats$sd_score))
cat(sprintf("Score range: %.3f to %.3f\n",
 overall_stats$min_score, overall_stats$max_score))
cat(sprintf("Median: %.3f, IQR: %.3f\n",
 overall_stats$median_score, overall_stats$iqr))
cat(sprintf("Cronbach's alpha: %.3f\n\n", cronbach_alpha))

cat("By PTAP type:\n")
for (i in 1:nrow(type_stats)) {
 cat(sprintf(" %s: mean = %.3f (SD = %.3f), n = %d\n",
 type_stats$type[i], type_stats$mean_score[i],
 type_stats$sd_score[i], type_stats$n[i]))
}

cat("\n" + paste(rep("=", 60), collapse = "") + "\n")
cat("Descriptive statistics complete!\n")

9. Cleanup

sessionInfo()

```

## 8. scripts/03\_pca\_factor\_analysis.R

```
#!/usr/bin/env Rscript
=====
03_pca_factor_analysis.R
Principal Components Analysis and Factor Analysis
Riyadh PTAP Audit Framework
=====
Author: Majed M Halawani
Description: Performs PCA with oblimin rotation and internal cross-validation
=====

1. Environment Setup

suppressPackageStartupMessages({
 library(tidyverse)
 library(here)
 library(yaml)
 library(psych)
 library(GPARotation)
 library(corrplot)
})

Load configuration
config <- yaml::read_yaml(here("config/config.yml"))
set.seed(config$sampling$seed)

Define paths
processed_path <- here(config$paths$processed_data)
output_path <- here(config$paths$outputs)

Create directories
dir.create(file.path(output_path, "tables"), recursive = TRUE, showWarnings = FALSE)
dir.create(file.path(output_path, "figures"), recursive = TRUE, showWarnings = FALSE)
dir.create(file.path(output_path, "supplementary"), recursive = TRUE, showWarnings = FALSE)
```

```

2. Load Data

cat("Loading data for PCA...\n")

audit_data <- read_csv(file.path(processed_path, "audit_data_with_scores.csv")
 show_col_types = FALSE)

indicator_names <- config$indicators$names
indicator_cols <- intersect(indicator_names, names(audit_data))

cat(sprintf(" Loaded %d PTAPs with %d indicators\n", nrow(audit_data), length(ir

3. Data Preparation for PCA

cat("\nPreparing data for PCA...\n")

3.1 Extract indicator matrix
X <- audit_data %>%
 select(all_of(indicator_cols)) %>%
 as.matrix()

3.2 Check for missing values
if (any(is.na(X))) {
 cat(" Warning: Missing values detected. Using pairwise complete observations.\n")
 X <- na.omit(X)
 cat(sprintf(" Remaining observations: %d\n", nrow(X)))
}

3.3 Standardize variables
X_scaled <- scale(X)

4. Assess Factorability

```

```

cat("\nAssessing factorability...\n")

4.1 Kaiser-Meyer-Olkin (KMO)
kmo_result <- psych::KMO(X_scaled)
cat(sprintf(" KMO overall: %.3f\n", kmo_result$MSA))
cat(sprintf(" KMO individual: %.3f - %.3f\n",
 min(kmo_result$MSAi), max(kmo_result$MSAi)))

4.2 Bartlett's test of sphericity
bartlett_result <- psych::cortest.bartlett(cor(X_scaled), n = nrow(X_scaled))
cat(sprintf(" Bartlett's test: $\chi^2 = %.1f$, df = %d, p = %.4f\n",
 bartlett_result$chisq, bartlett_result$df, bartlett_result$p.value))

5. Principal Components Analysis

cat("\nPerforming PCA...\n")

5.1 Initial PCA
pca_result <- psych::principal(X_scaled,
 nfactors = ncol(X_scaled),
 rotate = "none")

5.2 Eigenvalues
eigenvalues <- pca_result$values
cat(" Eigenvalues:\n")
for (i in 1:min(10, length(eigenvalues))) {
 cat(sprintf(" PC%d: %.3f (%.1f%% variance)\n",
 i, eigenvalues[i], eigenvalues[i] / sum(eigenvalues) * 100))
}

5.3 Parallel analysis
cat("\nPerforming parallel analysis...\n")
parallel_result <- psych::fa.parallel(X_scaled,
 n.iter = 1000,
 fa = "pc",

```

```

 show.legend = FALSE,
 main = "Parallel Analysis")

5.4 Determine number of factors
Using Kaiser criterion (eigenvalues > 1)
n_factors_kaiser <- sum(eigenvalues > 1)
cat(sprintf(" Kaiser criterion: %d factors\n", n_factors_kaiser))

Using parallel analysis
n_factors_parallel <- parallel_result$nfact
cat(sprintf(" Parallel analysis: %d factors\n", n_factors_parallel))

Using config
n_factors <- config$statistics$pca$factor_retention_criterion == "parallel_analys
 n_factors_parallel : n_factors_kaiser
cat(sprintf(" Final number of factors: %d\n", n_factors))

6. PCA with Oblimin Rotation

cat("\nPerforming PCA with oblimin rotation...\n")

6.1 Rotated PCA
pca_rotated <- psych::principal(X_scaled,
 nfactors = n_factors,
 rotate = "oblimin")

6.2 Extract loadings
loadings_matrix <- pca_rotated$loadings
cat(sprintf(" Rotation completed. Explained variance: %.1f%%\n",
 sum(pca_rotated$values[1:n_factors]) / sum(pca_rotated$values) * 100))

6.3 Factor correlations
factor_cors <- pca_rotated$Phi
cat(" Factor correlations:\n")
print(factor_cors)

```

```

7. Internal Cross-Validation (Split-Sample)

cat("\nPerforming split-sample internal cross-validation...\n")

7.1 Split sample into two halves
set.seed(config$sampling$seed)
n <- nrow(X_scaled)
half_size <- floor(n / 2)

Randomly select half of the observations
indices <- sample(1:n, half_size)
X_half1 <- X_scaled[indices,]
X_half2 <- X_scaled[-indices,]

cat(sprintf(" Half 1: %d observations\n", nrow(X_half1)))
cat(sprintf(" Half 2: %d observations\n", nrow(X_half2)))

7.2 Run PCA on each half
pca_half1 <- psych::principal(X_half1, nfactors = n_factors, rotate = "oblimin")
pca_half2 <- psych::principal(X_half2, nfactors = n_factors, rotate = "oblimin")

7.3 Calculate Tucker's congruence coefficient
Function to compute congruence between factor loadings
compute_tucker <- function(loadings1, loadings2) {
 # Ensure same number of factors
 n_fact <- min(ncol(loadings1), ncol(loadings2))
 loadings1 <- loadings1[, 1:n_fact]
 loadings2 <- loadings2[, 1:n_fact]

 congruence <- numeric(n_fact)
 for (k in 1:n_fact) {
 a <- loadings1[, k]
 b <- loadings2[, k]
 congruence[k] <- sum(a * b) / sqrt(sum(a^2) * sum(b^2))
 }
 return(congruence)
}

```

```

}

tucker_values <- compute_tucker(pca_half1$loadings, pca_half2$loadings)
cat(" Tucker's congruence coefficients:\n")
for (i in 1:length(tucker_values)) {
 cat(sprintf(" Factor %d: %.3f\n", i, tucker_values[i]))
}
cat(sprintf(" Overall Tucker's congruence: %.3f\n", mean(tucker_values)))

8. Bootstrap Stability Analysis

cat("\nPerforming bootstrap stability analysis...\n")

n_bootstrap <- config$statistics$bootstrap_iterations
cat(sprintf(" Bootstrap iterations: %d\n", n_bootstrap))

Function to compute bootstrap loadings
bootstrap_loadings <- matrix(NA, nrow = n_bootstrap, ncol = length(indicator_cc

for (b in 1:n_bootstrap) {
 if (b %% 100 == 0) cat(sprintf(" Iteration %d/%d\n", b, n_bootstrap))

 # Bootstrap sample
 boot_indices <- sample(1:n, n, replace = TRUE)
 X_boot <- X_scaled[boot_indices,]

 # Run PCA
 pca_boot <- psych::principal(X_boot, nfactors = n_factors, rotate = "oblimin")

 # Store loadings
 loadings_flat <- as.vector(pca_boot$loadings[, 1:n_factors])
 bootstrap_loadings[b,] <- loadings_flat
}

8.1 Calculate confidence intervals
loadings_flat <- as.vector(pca_rotated$loadings[, 1:n_factors])

```

```
ci_lower <- apply(bootstrap_loadings, 2, function(x) quantile(x, 0.025, na.rm = TR
ci_upper <- apply(bootstrap_loadings, 2, function(x) quantile(x, 0.975, na.rm = TR
```

```
cat(" Bootstrap confidence intervals calculated\n")
```

```

9. Create Output Tables

```

```
cat("\nCreating output tables...\n")
```

```
9.1 Loadings table with confidence intervals
```

```
loadings_table <- data.frame(
 indicator = rep(indicator_cols, n_factors),
 factor = rep(paste0("F", 1:n_factors), each = length(indicator_cols)),
 loading = loadings_flat,
 ci_lower = ci_lower,
 ci_upper = ci_upper
) %>%
 mutate(
 significant = !(ci_lower < 0 & ci_upper > 0),
 loading_formatted = sprintf("%.3f [%.3f-%.3f]", loading, ci_lower, ci_upper)
)
```

```
9.2 Factor correlation table
```

```
factor_cor_table <- as.data.frame(factor_cors)
colnames(factor_cor_table) <- paste0("F", 1:n_factors)
rownames(factor_cor_table) <- paste0("F", 1:n_factors)
```

```
9.3 Variance explained table
```

```
variance_table <- data.frame(
 factor = paste0("F", 1:n_factors),
 eigenvalue = pca_rotated$values[1:n_factors],
 variance = pca_rotated$values[1:n_factors] / sum(pca_rotated$values) * 100,
 cumulative = cumsum(pca_rotated$values[1:n_factors] / sum(pca_rotated$values)
)
```

```
9.4 Factor labels
```

```

factor_labels <- c(
 "Quality of Access",
 "Footpath Space",
 "Crossing Time",
 "Ease of Access",
 "Contextual Quality"
)
factor_labels <- factor_labels[1:n_factors]

10. Save Results

cat("\nSaving results...\n")

10.1 Save tables
write_csv(loadings_table, file.path(output_path, "tables/table_8_pca_loadings.csv"))
write_csv(factor_cor_table, file.path(output_path, "tables/table_9_factor_corr.csv"))
write_csv(variance_table, file.path(output_path, "tables/pca_variance_explained.csv"))

10.2 Save supplementary cross-validation results
cv_results <- data.frame(
 factor = paste0("F", 1:length(tucker_values)),
 tucker_congruence = tucker_values,
 classification = ifelse(tucker_values > 0.90, "Excellent",
 ifelse(tucker_values > 0.85, "Good", "Needs Review"))
)
write_csv(cv_results, file.path(output_path, "supplementary/supplementary_tab.csv"))

10.3 Save bootstrap summary
bootstrap_summary <- data.frame(
 parameter = c("iterations", "mean_se", "ci_width_mean"),
 value = c(n_bootstrap, mean(ci_upper - ci_lower), NA)
)
write_csv(bootstrap_summary, file.path(output_path, "supplementary/bootstrap_summary.csv"))

10.4 Save KMO and Bartlett results
factorability <- data.frame(

```

```

test = c("KMO", "Bartlett_chi2", "Bartlett_df", "Bartlett_p"),
value = c(kmo_result$MSA, bartlett_result$chisq, bartlett_result$df, bartlett_re
)
write_csv(factorability, file.path(output_path, "tables/factorability_tests.csv"))

11. Generate Figures

cat("\nGenerating figures...\n")

11.1 Scree plot
png(file.path(output_path, "figures/scree_plot.png"),
 width = 8, height = 6, units = "in", res = 300)

plot(eigenvalues, type = "b", pch = 19,
 xlab = "Component Number", ylab = "Eigenvalue",
 main = "Scree Plot with Parallel Analysis")
abline(h = 1, col = "red", lty = 2)
abline(h = parallel_result$pc.sim$mean, col = "blue", lty = 3)
legend("topright",
 legend = c("Eigenvalues", "Kaiser ($\lambda=1$)", "Parallel Analysis"),
 col = c("black", "red", "blue"),
 lty = c(1, 2, 3))
dev.off()
cat(" Scree plot saved\n")

11.2 Factor loading plot (heatmap)
loadings_matrix_plot <- as.matrix(pca_rotated$loadings[, 1:n_factors])
rownames(loadings_matrix_plot) <- indicator_cols
colnames(loadings_matrix_plot) <- factor_labels[1:n_factors]

png(file.path(output_path, "figures/factor_loadings_heatmap.png"),
 width = 10, height = 8, units = "in", res = 300)
corrplot(loadings_matrix_plot,
 is.corr = FALSE,
 method = "color",
 tl.col = "black",

```

```

 tl.srt = 45,
 col = colorRampPalette(c("red", "white", "blue"))(200),
 cl.lim = c(-1, 1))
dev.off()
cat(" Factor loadings heatmap saved\n")

11.3 Tucker's congruence plot
tucker_df <- data.frame(
 factor = paste0("F", 1:length(tucker_values)),
 congruence = tucker_values
)

png(file.path(output_path, "figures/figure_6_tucker_congruence.png"),
 width = 8, height = 6, units = "in", res = 300)

ggplot(tucker_df, aes(x = factor, y = congruence)) +
 geom_bar(stat = "identity", fill = "steelblue") +
 geom_hline(yintercept = 0.90, color = "red", linetype = "dashed") +
 geom_hline(yintercept = 0.85, color = "orange", linetype = "dotted") +
 ylim(0, 1) +
 labs(title = "Tucker's Congruence Coefficient",
 x = "Factor",
 y = "Congruence") +
 theme_minimal() +
 theme(
 plot.title = element_text(hjust = 0.5, face = "bold"),
 axis.text = element_text(size = 12),
 axis.title = element_text(size = 14)
)

dev.off()
cat(" Tucker's congruence plot saved\n")

12. Summary Report

cat("\n" + paste(rep("=", 60), collapse = "") + "\n")

```

```

cat("PCA FACTOR ANALYSIS SUMMARY REPORT\n")
cat(paste(rep("=", 60), collapse = "") + "\n\n")

cat("Factorability Assessment:\n")
cat(sprintf(" KMO: %.3f\n", kmo_result$MSA))
cat(sprintf(" Bartlett's test: $\chi^2 = %.1f$, $p = %.4f$ \n",
 bartlett_result$chisq, bartlett_result$p.value))
cat("\n")

cat(sprintf("Number of factors extracted: %d\n", n_factors))
cat(sprintf("Total variance explained: %.1f%%\n",
 sum(pca_rotated$values[1:n_factors]) / sum(pca_rotated$values) * 100))
cat("\n")

cat("Factor Labels:\n")
for (i in 1:length(factor_labels)) {
 cat(sprintf(" %s: %s\n", paste0("F", i), factor_labels[i]))
}
cat("\n")

cat("Cross-validation:\n")
cat(sprintf(" Tucker's congruence coefficients: %.3f - %.3f\n",
 min(tucker_values), max(tucker_values)))
cat(sprintf(" Overall: %.3f\n", mean(tucker_values)))
if (mean(tucker_values) > 0.90) {
 cat(" Status: EXCELLENT agreement\n")
} else if (mean(tucker_values) > 0.85) {
 cat(" Status: GOOD agreement\n")
} else {
 cat(" Status: Needs review\n")
}
cat("\n")

cat("Bootstrap Stability:\n")
cat(sprintf(" Iterations: %d\n", n_bootstrap))
cat(sprintf(" Mean CI width: %.3f\n", mean(ci_upper - ci_lower)))
cat("\n")

```

```
cat(paste(rep("=", 60), collapse = "") + "\n")
cat("PCA analysis complete!\n")
```

```

13. Cleanup

sessionInfo()
```

---

## 9. scripts/04\_spatial\_analysis.R

```
#!/usr/bin/env Rscript
=====
04_spatial_analysis.R
Spatial Analysis Script
Riyadh PTAP Audit Framework
=====
Author: Majed M Halawani
Description: Performs spatial autocorrelation analysis (Moran's I, LISA, Getis-Ord (
=====

1. Environment Setup

suppressPackageStartupMessages({
 library(tidyverse)
 library(sf)
 library(here)
 library(yaml)
 library(spdep)
 library(spatstat)
 library(ggplot2)
 library(viridis)
})

Load configuration
```

```

config <- yaml::read_yaml(here("config/config.yml"))
set.seed(config$sampling$seed)

Define paths
processed_path <- here(config$paths$processed_data)
output_path <- here(config$paths$outputs)

Create directories
dir.create(file.path(output_path, "tables"), recursive = TRUE, showWarnings = FALSE)
dir.create(file.path(output_path, "figures"), recursive = TRUE, showWarnings = FALSE)
dir.create(file.path(output_path, "supplementary"), recursive = TRUE, showWarnings = FALSE)

2. Load Data

cat("Loading data for spatial analysis...\n")

Load audit data with scores
audit_data <- read_csv(file.path(processed_path, "audit_data_with_scores.csv"),
 show_col_types = FALSE)

Load PTAP locations
all_ptaps <- read_csv(file.path(processed_path, "all_ptaps_clean.csv"),
 show_col_types = FALSE)

Load ward boundaries
ward_file <- file.path(processed_path, "ward_boundaries_clean.shp")
if (file.exists(ward_file)) {
 wards <- st_read(ward_file, quiet = TRUE)
 cat(" Ward boundaries loaded\n")
} else {
 cat(" WARNING: Ward boundaries not found. Creating synthetic boundaries.\n")
 # Create synthetic boundaries for testing
 wards <- NULL
}

```

### *# 3. Prepare Spatial Data*

*# -----*

```
cat("\nPreparing spatial data...\n")
```

#### *# 3.1 Merge audit scores with PTAP locations*

```
ptap_spatial <- all_ptaps %>%
 left_join(audit_data %>% select(ptap_id, composite_score, type),
 by = "ptap_id") %>%
 filter(!is.na(composite_score))
```

```
cat(sprintf(" %d PTAPs with spatial data\n", nrow(ptap_spatial)))
```

#### *# 3.2 Convert to sf object*

```
ptap_sf <- st_as_sf(ptap_spatial,
 coords = c("longitude", "latitude"),
 crs = 4326)
```

#### *# 3.3 Project to metric CRS for distance calculations*

```
ptap_sf_proj <- st_transform(ptap_sf, 3857)
```

*# -----*

### *# 4. Calculate Kernel Density*

*# -----*

```
cat("\nCalculating kernel density...\n")
```

#### *# 4.1 Extract coordinates*

```
coords <- st_coordinates(ptap_sf_proj)
```

#### *# 4.2 Create point pattern*

```
if (nrow(coords) > 0) {
 # Create bounding window
 x_range <- range(coords[, "X"])
 y_range <- range(coords[, "Y"])
 win <- owin(xrange = x_range, yrange = y_range)
```

```
Create point pattern
```

```

ppp <- ppp(x = coords[, "X"], y = coords[, "Y"], window = win)

Calculate kernel density
bandwidth <- config$spatial$kernel_bandwidth # meters
density_est <- density(ppp, sigma = bandwidth, eps = 100)

cat(" Kernel density calculated\n")
}

5. Spatial Weights Matrix

cat("\nCreating spatial weights matrix...\n")

5.1 Create neighbors list using distance threshold
coords_matrix <- st_coordinates(ptap_sf_proj)
dist_threshold <- config$spatial$inverse_distance_threshold_km * 1000 # Convert to meters

Create k-nearest neighbors (k=5 for robustness)
nb <- knn2nb(knearneigh(coords_matrix, k = 5))

Create weights list
listw <- nb2listw(nb, style = "W")

cat(sprintf(" Created weights matrix with %d neighbors\n", length(nb)))

5.2 Alternative weight matrices for sensitivity analysis
Queen contiguity (if polygon data available)
if (!is.null(wards)) {
 # Convert PTAPs to points within wards
 ptap_in_wards <- st_join(ptap_sf, wards)

 # Create queen contiguity weights
 nb_queen <- poly2nb(wards, queen = TRUE)
 listw_queen <- nb2listw(nb_queen, style = "W")
 cat(" Queen contiguity weights created\n")
} else {

```

```

listw_queen <- NULL
}

Inverse distance weights
dist_matrix <- as.matrix(dist(coords_matrix))
inv_dist <- 1 / dist_matrix
diag(inv_dist) <- 0
listw_inv <- mat2listw(inv_dist, style = "W")

cat(" Inverse distance weights created\n")

6. Global Moran's I

cat("\nCalculating Global Moran's I...\n")

6.1 Moran's I using k-nearest weights
moran_result <- moran.test(ptap_sf_proj$composite_score, listw,
 randomisation = TRUE)

moran_i <- moran_result$estimate[1]
moran_p <- moran_result$p.value

cat(sprintf(" Moran's I: %.3f\n", moran_i))
cat(sprintf(" Expected I: %.3f\n", moran_result$estimate[2]))
cat(sprintf(" p-value: %.4f\n", moran_p))

6.2 Permutation test for significance
moran_perm <- moran.mc(ptap_sf_proj$composite_score, listw, nsim = 999)
cat(sprintf(" Permutation test p-value: %.4f\n", moran_perm$p.value))

6.3 Moran plot
moran_plot_data <- moran.plot(ptap_sf_proj$composite_score, listw)

7. Local Indicators of Spatial Association (LISA)

```

```

cat("\nCalculating LISA...\n")

7.1 Local Moran's I
lisa_result <- localmoran(ptap_sf_proj$composite_score, listw)

7.2 Extract LISA statistics
ptap_sf_proj$local_moran <- lisa_result[, 1]
ptap_sf_proj$local_p <- lisa_result[, 5]

7.3 Classify clusters
ptap_sf_proj <- ptap_sf_proj %>%
 mutate(
 lisa_class = case_when(
 local_p > 0.05 ~ "Not Significant",
 local_moran > 0 & composite_score > mean(composite_score) ~ "High-High",
 local_moran > 0 & composite_score < mean(composite_score) ~ "Low-Low",
 local_moran < 0 & composite_score > mean(composite_score) ~ "High-Low",
 local_moran < 0 & composite_score < mean(composite_score) ~ "Low-High",
 TRUE ~ "Not Significant"
)
)

7.4 Count clusters
cluster_counts <- ptap_sf_proj %>%
 group_by(lisa_class) %>%
 summarise(count = n()) %>%
 arrange(desc(count))

cat(" LISA cluster counts:\n")
for (i in 1:nrow(cluster_counts)) {
 cat(sprintf(" %s: %d\n", cluster_counts$lisa_class[i], cluster_counts$count[i]))
}

7.5 Identify low-low clusters (peripheral)
low_low_ptaps <- ptap_sf_proj %>%
 filter(lisa_class == "Low-Low")

```

```

cat(sprintf(" Low-Low clusters: %d PTAPs\n", nrow(low_low_ptaps)))

8. Getis-Ord Gi* Analysis

cat("\nCalculating Getis-Ord Gi*...\n")

8.1 Getis-Ord Gi* for different search windows
gi_results <- list()

for (window in config$spatial$getis_ord_windows) {
 window_km <- window * 1000 # Convert to meters

 # Create distance-based weights for this window
 nb_gi <- dnearneigh(coords_matrix, 0, window_km)
 listw_gi <- nb2listw(nb_gi, style = "B")

 # Calculate Gi*
 gi_result <- localG(ptap_sf_proj$composite_score, listw_gi)

 # Add to results
 gi_results[[paste0(window, "km")]] <- gi_result

 # Count hot and cold spots
 hot_count <- sum(gi_result > 1.96, na.rm = TRUE)
 cold_count <- sum(gi_result < -1.96, na.rm = TRUE)

 cat(sprintf(" Window %dkm: Hot=%d, Cold=%d\n", window, hot_count, cold_co
}

8.2 Use 2km window for main results
gi_window <- config$spatial$getis_ord_windows[2] * 1000
nb_gi_main <- dnearneigh(coords_matrix, 0, gi_window)
listw_gi_main <- nb2listw(nb_gi_main, style = "B")
gi_main <- localG(ptap_sf_proj$composite_score, listw_gi_main)

ptap_sf_proj$gi_score <- gi_main

```

```

ptap_sf_proj$gi_significant <- abs(gi_main) > 1.96

8.3 Classify hot/cold spots
ptap_sf_proj <- ptap_sf_proj %>%
 mutate(
 gi_class = case_when(
 gi_score > 1.96 ~ "Hot Spot",
 gi_score < -1.96 ~ "Cold Spot",
 TRUE ~ "Not Significant"
)
)

8.4 Count spots
spot_counts <- ptap_sf_proj %>%
 group_by(gi_class) %>%
 summarise(count = n()) %>%
 arrange(desc(count))

cat(" Getis-Ord Gi* counts:\n")
for (i in 1:nrow(spot_counts)) {
 cat(sprintf(" %s: %d\n", spot_counts$gi_class[i], spot_counts$count[i]))
}

9. Calculate Spatial Extent of Clusters

cat("\nCalculating spatial extent of clusters...\n")

9.1 Create convex hulls for hot and cold spots
hot_pts <- ptap_sf_proj %>% filter(gi_class == "Hot Spot")
cold_pts <- ptap_sf_proj %>% filter(gi_class == "Cold Spot")

if (nrow(hot_pts) >= 3) {
 hot_hull <- st_convex_hull(st_union(hot_pts))
 hot_area <- st_area(hot_hull) / 1e6 # km2
} else {
 hot_area <- 0

```

```

}

if (nrow(cold_pts) >= 3) {
 cold_hull <- st_convex_hull(st_union(cold_pts))
 cold_area <- st_area(cold_hull) / 1e6 # km2
} else {
 cold_area <- 0
}

cat(sprintf(" Hot spot area: %.1f km2\n", hot_area))
cat(sprintf(" Cold spot area: %.1f km2\n", cold_area))

10. Save Results

cat("\nSaving results...\n")

10.1 Save Moran's I results
moran_summary <- data.frame(
 metric = c("Moran_I", "Expected_I", "Variance", "p_value", "p_value_perm"),
 value = c(moran_i, moran_result$estimate[2], moran_result$estimate[3],
 moran_p, moran_perm$p.value)
)
write_csv(moran_summary, file.path(output_path, "tables/moran_i_results.csv"))

10.2 Save LISA results
lisa_summary <- ptap_sf_proj %>%
 st_drop_geometry() %>%
 select(ptap_id, composite_score, local_moran, local_p, lisa_class) %>%
 arrange(desc(composite_score))

write_csv(lisa_summary, file.path(output_path, "tables/lisa_results.csv"))

10.3 Save cluster counts
write_csv(cluster_counts, file.path(output_path, "tables/lisa_cluster_counts.csv"))

10.4 Save Getis-Ord results

```

```

gi_summary <- ptap_sf_proj %>%
 st_drop_geometry() %>%
 select(ptap_id, composite_score, gi_score, gi_class) %>%
 arrange(desc(composite_score))

write_csv(gi_summary, file.path(output_path, "tables/getis_ord_results.csv"))

10.5 Save supplementary tables
Gi* results for all windows
gi_all_windows <- data.frame()
for (w in names(gi_results)) {
 gi_temp <- data.frame(
 ptap_id = ptap_sf_proj$ptap_id,
 window = w,
 gi_score = gi_results[[w]]
)
 gi_all_windows <- bind_rows(gi_all_windows, gi_temp)
}
write_csv(gi_all_windows, file.path(output_path, "supplementary/supplementary

10.6 Sensitivity analysis for spatial weights
weight_sensitivity <- data.frame(
 weight_type = c("KNN", "Inverse_Distance", "Queen_Contiguity"),
 moran_i = c(moran_i, NA, NA),
 p_value = c(moran_p, NA, NA)
)

Calculate Moran's I for inverse distance
moran_inv <- moran.test(ptap_sf_proj$composite_score, listw_inv, randomisati
weight_sensitivity[2, "moran_i"] <- moran_inv$estimate[1]
weight_sensitivity[2, "p_value"] <- moran_inv$p.value

Calculate Moran's I for queen contiguity (if available)
if (!is.null(listw_queen)) {
 moran_queen <- moran.test(ptap_sf_proj$composite_score, listw_queen, rand
weight_sensitivity[3, "moran_i"] <- moran_queen$estimate[1]
weight_sensitivity[3, "p_value"] <- moran_queen$p.value
}

```

```

write_csv(weight_sensitivity, file.path(output_path, "supplementary/supplementa

10.7 Save spatial extent summary
extent_summary <- data.frame(
 cluster_type = c("Hot_Spots", "Cold_Spots"),
 area_km2 = c(hot_area, cold_area),
 ptap_count = c(nrow(hot_pts), nrow(cold_pts))
)
write_csv(extent_summary, file.path(output_path, "tables/spatial_extent_summ

11. Generate Figures

cat("\nGenerating figures...\n")

11.1 LISA Cluster Map
ptap_sf_proj$lisa_class_ordered <- factor(ptap_sf_proj$lisa_class,
 levels = c("High-High", "Low-Low",
 "High-Low", "Low-High",
 "Not Significant"))

png(file.path(output_path, "figures/figure_8_lisa_cluster.png"),
 width = 10, height = 8, units = "in", res = 300)

Plot using ggplot
cluster_colors <- c("High-High" = "red",
 "Low-Low" = "blue",
 "High-Low" = "orange",
 "Low-High" = "purple",
 "Not Significant" = "gray80")

ggplot(ptap_sf_proj) +
 # Base map (if ward boundaries available)
 {if (!is.null(wards)) geom_sf(data = wards, fill = "gray95", color = "gray70")} +
 # PTAP points
 geom_sf(aes(color = lisa_class_ordered, shape = type), size = 3) +

```

```

scale_color_manual(values = cluster_colors, name = "LISA Cluster") +
scale_shape_manual(values = c("metro" = 16, "bus" = 17), name = "PTAP Type") +
labs(title = "LISA Cluster Map of Infrastructure Quality Scores",
 subtitle = "High-High clusters in central areas, Low-Low clusters in peripheral",
 theme_minimal() +
 theme(
 plot.title = element_text(hjust = 0.5, face = "bold"),
 plot.subtitle = element_text(hjust = 0.5, color = "gray50"),
 legend.position = "right"
)
)

dev.off()
cat(" LISA cluster map saved\n")

11.2 Moran's I scatter plot
moran_scatter_data <- moran.plot(ptap_sf_proj$composite_score, listw,
 pch = 19, col = "black",
 xlab = "Composite Score",
 ylab = "Spatial Lag",
 main = "Moran's I Scatter Plot")

Save as PNG
png(file.path(output_path, "figures/moran_scatter_plot.png"),
 width = 8, height = 6, units = "in", res = 300)

plot(moran_scatter_data, pch = 19,
 xlab = "Composite Score",
 ylab = "Spatial Lag",
 main = sprintf("Moran's I = %.3f (p = %.4f)", moran_i, moran_p))

dev.off()
cat(" Moran scatter plot saved\n")

11.3 Getis-Ord Gi* hot spot map
png(file.path(output_path, "figures/getis_ord_hotspots.png"),
 width = 10, height = 8, units = "in", res = 300)

gi_colors <- c("Hot Spot" = "red",

```

```

 "Cold Spot" = "blue",
 "Not Significant" = "gray80")

ggplot(ptap_sf_proj) +
 {if (!is.null(wards)) geom_sf(data = wards, fill = "gray95", color = "gray70")} +
 geom_sf(aes(color = gi_class, shape = type), size = 3) +
 scale_color_manual(values = gi_colors, name = "Getis-Ord Gi*") +
 scale_shape_manual(values = c("metro" = 16, "bus" = 17), name = "PTAP Type") +
 labs(title = "Getis-Ord Gi* Hot Spot Analysis",
 subtitle = "2km search window") +
 theme_minimal() +
 theme(
 plot.title = element_text(hjust = 0.5, face = "bold"),
 plot.subtitle = element_text(hjust = 0.5, color = "gray50")
)

dev.off()
cat(" Getis-Ord map saved\n")

11.4 PTAP Density Map (Figure 7)
density_data <- ptap_sf_proj %>%
 mutate(ward = as.character(ward_id))

Calculate density by ward (if ward data available)
if (!is.null(wards)) {
 ward_density <- st_join(wards, ptap_sf_proj) %>%
 group_by(ward_id) %>%
 summarise(
 ptap_count = n(),
 area_km2 = first(area_km2),
 density = ptap_count / area_km2
)

png(file.path(output_path, "figures/figure_7_ward_density.png"),
 width = 10, height = 8, units = "in", res = 300)

ggplot(ward_density) +
 geom_sf(aes(fill = density), color = "gray30") +

```

```

scale_fill_viridis_c(name = "PTAPs per km²", option = "magma") +
labs(title = "Ward-Level PTAP Density",
 subtitle = "Strong central-peripheral gradient") +
theme_minimal() +
theme(
 plot.title = element_text(hjust = 0.5, face = "bold"),
 plot.subtitle = element_text(hjust = 0.5, color = "gray50")
)

dev.off()
cat(" Ward density map saved\n")
}

12. Summary Report

cat("\n" + paste(rep("=", 60), collapse = "") + "\n")
cat("SPATIAL ANALYSIS SUMMARY REPORT\n")
cat(paste(rep("=", 60), collapse = "") + "\n\n")

cat("Global Spatial Autocorrelation:\n")
cat(sprintf(" Moran's I: %.3f\n", moran_i))
cat(sprintf(" Expected I: %.3f\n", moran_result$estimate[2]))
cat(sprintf(" p-value: %.4f\n", moran_p))
cat("\n")

cat("LISA Analysis:\n")
for (i in 1:nrow(cluster_counts)) {
 cat(sprintf(" %s: %d PTAPs\n", cluster_counts$lisa_class[i], cluster_counts$cou
})
cat("\n")

cat("Getis-Ord Gi* Analysis:\n")
for (i in 1:nrow(spot_counts)) {
 cat(sprintf(" %s: %d PTAPs\n", spot_counts$gi_class[i], spot_counts$count[i]))
}
cat("\n")

```

```

cat("Spatial Extent:\n")
cat(sprintf(" Hot spots: %.1f km² (%d PTAPs)\n", hot_area, nrow(hot_pts)))
cat(sprintf(" Cold spots: %.1f km² (%d PTAPs)\n", cold_area, nrow(cold_pts)))
cat("\n")

cat(paste(rep(" ", 60), collapse = "") + "\n")
cat("Spatial analysis complete!\n")

13. Cleanup

sessionInfo()

```

---

## 10. scripts/05\_regression\_analysis.R

```

#!/usr/bin/env Rscript
=====
05_regression_analysis.R
Regression Analysis Script
Riyadh PTAP Audit Framework
=====
Author: Majed M Halawani
Description: Performs multiple linear regression for proxy-based assessment
=====

1. Environment Setup

suppressPackageStartupMessages({
 library(tidyverse)
 library(here)
 library(yaml)
 library(car)
 library(lmtest)

```

```

library(performance)
library(broom)
library(sjPlot)
})

Load configuration
config <- yaml::read_yaml(here("config/config.yml"))
set.seed(config$sampling$seed)

Define paths
processed_path <- here(config$paths$processed_data)
output_path <- here(config$paths$outputs)

Create directories
dir.create(file.path(output_path, "tables"), recursive = TRUE, showWarnings = FALSE)
dir.create(file.path(output_path, "figures"), recursive = TRUE, showWarnings = FALSE)
dir.create(file.path(output_path, "supplementary"), recursive = TRUE, showWarnings = FALSE)

2. Load Data

cat("Loading data for regression analysis...\n")

Load audit data with scores
audit_data <- read_csv(file.path(processed_path, "audit_data_with_scores.csv"),
 show_col_types = FALSE)

Load PTAP data
ptap_data <- read_csv(file.path(processed_path, "all_ptaps_clean.csv"),
 show_col_types = FALSE)

Load ward data if available
ward_file <- file.path(processed_path, "ward_boundaries_clean.shp")
if (file.exists(ward_file)) {
 ward_data <- st_read(ward_file, quiet = TRUE)
 cat(" Ward data loaded\n")
} else {

```

```

ward_data <- NULL
}

3. Prepare Data for Regression

cat("\nPreparing data for regression...\n")

3.1 Merge datasets
reg_data <- audit_data %>%
 left_join(ptap_data, by = c("ptap_id", "type")) %>%
 mutate(
 # Define central/peripheral classification
 is_central = ifelse(density_stratum %in% c("high", "medium"), 1, 0),
 # Ridership density categories
 ridership_high = ifelse(density_stratum == "high", 1, 0),
 ridership_medium = ifelse(density_stratum == "medium", 1, 0),
 # Type dummy
 is_metro = ifelse(type == "metro", 1, 0)
)

cat(sprintf(" Regression data: %d observations\n", nrow(reg_data)))

3.2 Define variables
independent_vars <- c("ridership_high", "ridership_medium", "is_metro", "is_central")
dependent_var <- "composite_score"

cat(" Variables:")
cat(sprintf(" Dependent: %s\n", dependent_var))
cat(sprintf(" Independent: %s\n", paste(independent_vars, collapse = ", ")))

4. Model Specification

cat("\nSpecifying regression models...\n")

```

```
4.1 Model 1: Main effects
```

```
model1 <- lm(composite_score ~ ridership_high + ridership_medium + is_metro +
 data = reg_data)
```

```
4.2 Model 2: With interaction (ridership × location)
```

```
model2 <- lm(composite_score ~ ridership_high + ridership_medium + is_metro +
 ridership_high:is_central + ridership_medium:is_central,
 data = reg_data)
```

```
4.3 Model 3: Full model with all interactions
```

```
model3 <- lm(composite_score ~ (ridership_high + ridership_medium + is_metro
 data = reg_data)
```

```

```

```
5. Model Diagnostics
```

```

```

```
cat("\nPerforming model diagnostics...\n")
```

```
5.1 Check multicollinearity (VIF)
```

```
vif_results <- vif(model2)
```

```
cat(" Variance Inflation Factors:\n")
```

```
for (i in 1:length(vif_results)) {
 cat(sprintf(" %s: %.3f\n", names(vif_results)[i], vif_results[i]))
}
```

```
5.2 Check heteroscedasticity
```

```
bp_test <- bptest(model2)
```

```
cat(sprintf(" Breusch-Pagan test: BP = %.3f, p = %.4f\n",
 bp_test$statistic, bp_test$p.value))
```

```
5.3 Check normality of residuals
```

```
residuals <- residuals(model2)
```

```
shapiro_test <- shapiro.test(residuals)
```

```
cat(sprintf(" Shapiro-Wilk test: W = %.4f, p = %.4f\n",
 shapiro_test$statistic, shapiro_test$p.value))
```

```
5.4 Other diagnostics
```

```

dw_test <- dwtest(model2)
cat(sprintf(" Durbin-Watson test: DW = %.3f, p = %.4f\n",
 dw_test$statistic, dw_test$p.value))

6. Model Selection

cat("\nComparing models...\n")

6.1 AIC/BIC comparison
model_comparison <- data.frame(
 model = c("Model 1", "Model 2", "Model 3"),
 AIC = c(AIC(model1), AIC(model2), AIC(model3)),
 BIC = c(BIC(model1), BIC(model2), BIC(model3)),
 R2_adj = c(summary(model1)$adj.r.squared,
 summary(model2)$adj.r.squared,
 summary(model3)$adj.r.squared)
)

cat(" Model comparison:\n")
print(model_comparison)

6.2 F-test for nested models
f_test <- anova(model1, model2)
cat(sprintf(" F-test (Model 1 vs Model 2): F = %.3f, p = %.4f\n",
 f_test$F[2], f_test$`Pr(>F)`[2]))

Select best model (Model 2 based on AIC and interpretability)
selected_model <- model2
cat(" Selected model: Model 2 (with ridership × location interaction)\n")

7. Extract Results

cat("\nExtracting model results...\n")

```

*# 7.1 Coefficient table*

```
coef_table <- tidy(selected_model, conf.int = TRUE) %>%
 mutate(
 sig = ifelse(p.value < 0.001, "***",
 ifelse(p.value < 0.01, "**",
 ifelse(p.value < 0.05, "*", "ns"))),
 formatted = sprintf("%.3f (%.3f, %.3f)", estimate, conf.low, conf.high)
)
```

*# 7.2 Model summary*

```
model_summary <- glance(selected_model)
```

*# 7.3 Calculate standardized coefficients*

```
library(QuantPsyc)
std_coef <- lm.beta(selected_model)
std_coef_df <- data.frame(
 variable = names(std_coef),
 std_coef = std_coef
)
```

```

```

*# 8. Subgroup Analysis*

```

```

```
cat("\nPerforming subgroup analysis...\n")
```

*# 8.1 Central vs Peripheral analysis*

```
central_data <- reg_data %>% filter(is_central == 1)
peripheral_data <- reg_data %>% filter(is_central == 0)
```

*# Regression in peripheral areas (for proxy-based assessment)*

```
model_peripheral <- lm(composite_score ~ ridership_high + ridership_medium +
 data = peripheral_data)
```

*# Regression in central areas*

```
model_central <- lm(composite_score ~ ridership_high + ridership_medium + is_
 data = central_data)
```

```
cat(" Peripheral areas model:\n")
summary_peripheral <- summary(model_peripheral)
cat(sprintf(" R2 adjusted: %.3f\n", summary_peripheral$adj.r.squared))
```

```
cat(" Central areas model:\n")
summary_central <- summary(model_central)
cat(sprintf(" R2 adjusted: %.3f\n", summary_central$adj.r.squared))
```

```
8.2 Extract ridership effects
```

```
ridership_peripheral <- tidy(model_peripheral) %>%
 filter(grepl("ridership", term))
```

```
ridership_central <- tidy(model_central) %>%
 filter(grepl("ridership", term))
```

```

9. Proxy-Based Assessment Results

```

```
cat("\nProxy-based assessment results:\n")
```

```
9.1 Main effect of ridership in peripheral areas
```

```
beta_ridership_peripheral <- coef(selected_model)["ridership_high"] +
 coef(selected_model)["ridership_high:is_central"]
```

```
cat(sprintf(" Ridership effect in peripheral areas: β = %.3f\n",
 beta_ridership_peripheral))
```

```
9.2 Check significance
```

```
if (summary_peripheral$coefficients[2, 4] < 0.05) {
 cat(" Status: SIGNIFICANT (p < 0.05) - supports construct validity\n")
} else {
 cat(" Status: Not significant\n")
}
```

```

10. Save Results

```

```

cat("\nSaving results...\n")

10.1 Save coefficient table
write_csv(coef_table, file.path(output_path, "tables/regression_coefficients.csv")

10.2 Save model summary
model_summary_df <- data.frame(
 metric = c("R2", "R2_adj", "sigma", "statistic", "p_value", "df", "AIC", "BIC"),
 value = c(model_summary$r.squared, model_summary$adj.r.squared,
 model_summary$sigma, model_summary$statistic,
 model_summary$p.value, model_summary$df,
 AIC(selected_model), BIC(selected_model))
)
write_csv(model_summary_df, file.path(output_path, "tables/regression_model_

10.3 Save standardized coefficients
write_csv(std_coef_df, file.path(output_path, "tables/regression_std_coefficien

10.4 Save diagnostics
diagnostics_df <- data.frame(
 test = c("VIF", "Breusch_Pagan", "Shapiro_Wilk", "Durbin_Watson"),
 statistic = c(mean(vif_results), bp_test$statistic, shapiro_test$statistic, dw_test
 p_value = c(NA, bp_test$p.value, shapiro_test$p.value, dw_test$p.value)
)
write_csv(diagnostics_df, file.path(output_path, "supplementary/supplementary_

10.5 Save subgroup analysis
subgroup_results <- bind_rows(
 tidy(model_peripheral) %>% mutate(subgroup = "Peripheral"),
 tidy(model_central) %>% mutate(subgroup = "Central")
)
write_csv(subgroup_results, file.path(output_path, "tables/regression_subgroup

10.6 Save full regression output (Supplementary Table S2)
regression_full <- data.frame(
 term = names(coef(selected_model)),
 estimate = coef(selected_model),

```

```

std_error = summary(selected_model)$coefficients[, 2],
t_value = summary(selected_model)$coefficients[, 3],
p_value = summary(selected_model)$coefficients[, 4]
)
write_csv(regression_full, file.path(output_path, "supplementary/supplementary

11. Generate Figures

cat("\nGenerating figures...\n")

11.1 Residual plots
png(file.path(output_path, "figures/regression_residual_plots.png"),
 width = 12, height = 8, units = "in", res = 300)

par(mfrow = c(2, 2))
plot(selected_model)

dev.off()
cat(" Residual plots saved\n")

11.2 Predicted vs Actual
reg_data$predicted <- predict(selected_model)

png(file.path(output_path, "figures/regression_predicted_vs_actual.png"),
 width = 8, height = 6, units = "in", res = 300)

ggplot(reg_data, aes(x = predicted, y = composite_score)) +
 geom_point(aes(color = type, shape = factor(is_central)), size = 3) +
 geom_abline(slope = 1, intercept = 0, color = "red", linetype = "dashed") +
 labs(title = "Predicted vs Actual Composite Scores",
 x = "Predicted Score", y = "Actual Score",
 color = "PTAP Type", shape = "Location") +
 theme_minimal() +
 theme(
 plot.title = element_text(hjust = 0.5, face = "bold"),
 legend.position = "right"
)

```

```

)

dev.off()
cat(" Predicted vs actual plot saved\n")

11.3 Coefficient plot
png(file.path(output_path, "figures/regression_coefficient_plot.png"),
 width = 10, height = 6, units = "in", res = 300)

coef_plot <- coef_table %>%
 filter(term != "(Intercept)") %>%
 mutate(
 term_label = case_when(
 term == "ridership_high" ~ "High Ridership",
 term == "ridership_medium" ~ "Medium Ridership",
 term == "is_metro" ~ "Metro (vs Bus)",
 term == "is_central" ~ "Central (vs Peripheral)",
 term == "ridership_high:is_central" ~ "High Ridership × Central",
 term == "ridership_medium:is_central" ~ "Medium Ridership × Central",
 TRUE ~ term
)
)

ggplot(coef_plot, aes(x = estimate, y = reorder(term_label, estimate))) +
 geom_vline(xintercept = 0, linetype = "dashed", color = "gray50") +
 geom_errorbarh(aes(xmin = conf.low, xmax = conf.high), height = 0.2, color = "steelblue") +
 geom_point(size = 3, color = "steelblue") +
 labs(title = "Regression Coefficient Estimates with 95% CI",
 x = "Estimate", y = "") +
 theme_minimal() +
 theme(
 plot.title = element_text(hjust = 0.5, face = "bold"),
 axis.text.y = element_text(size = 11)
)

dev.off()
cat(" Coefficient plot saved\n")

```

```

12. Summary Report

cat("\n" + paste(rep("=", 60), collapse = "") + "\n")
cat("REGRESSION ANALYSIS SUMMARY REPORT\n")
cat(paste(rep("=", 60), collapse = "") + "\n\n")

cat("Model Fit:\n")
cat(sprintf(" R²: %.3f\n", model_summary$r.squared))
cat(sprintf(" Adjusted R²: %.3f\n", model_summary$adj.r.squared))
cat(sprintf(" F-statistic: %.3f (p = %.4f)\n",
 model_summary$statistic, model_summary$p.value))
cat("\n")

cat("Key Coefficients:\n")
for (i in 1:nrow(coef_table)) {
 if (coef_table$term[i] != "(Intercept)") {
 sig <- ifelse(coef_table$sig[i] == "***", " (p < 0.001)",
 ifelse(coef_table$sig[i] == "**", " (p < 0.01)",
 ifelse(coef_table$sig[i] == "*", " (p < 0.05)", " (ns)")))
 cat(sprintf(" %s: %.3f%s\n", coef_table$term[i], coef_table$estimate[i], sig))
 }
}
cat("\n")

cat("Proxy-Based Assessment:\n")
cat(sprintf(" Ridership effect in peripheral areas: β = %.3f\n",
 beta_ridership_peripheral))
if (summary_peripheral$coefficients[2, 4] < 0.05) {
 cat(" Status: SIGNIFICANT - provides construct validity evidence\n")
} else {
 cat(" Status: Not significant\n")
}
cat("\n")

cat("Diagnostics:\n")
cat(sprintf(" Mean VIF: %.3f\n", mean(vif_results)))

```

```
cat(sprintf(" Breusch-Pagan: p = %.4f\n", bp_test$p.value))
cat(sprintf(" Shapiro-Wilk: p = %.4f\n", shapiro_test$p.value))
cat("\n")
```

```
cat(paste(rep("=", 60), collapse = "") + "\n")
cat("Regression analysis complete!\n")
```

```

13. Cleanup

```

```
sessionInfo()
```

---

## 11. scripts/06\_latent\_class\_analysis.R

```
#!/usr/bin/env Rscript
=====
06_latent_class_analysis.R
Latent Class Analysis Script
Riyadh PTAP Audit Framework
=====
Author: Majed M Halawani
Description: Identifies PTAP typologies using latent class analysis
=====

1. Environment Setup

suppressPackageStartupMessages({
 library(tidyverse)
 library(here)
 library(yaml)
 library(poLCA)
 library(ggplot2)
 library(gridExtra)
 library(factoextra)
```

```

})

Load configuration
config <- yaml::read_yaml(here("config/config.yml"))
set.seed(config$sampling$seed)

Define paths
processed_path <- here(config$paths$processed_data)
output_path <- here(config$paths$outputs)

Create directories
dir.create(file.path(output_path, "tables"), recursive = TRUE, showWarnings = FALSE)
dir.create(file.path(output_path, "figures"), recursive = TRUE, showWarnings = FALSE)
dir.create(file.path(output_path, "supplementary"), recursive = TRUE, showWarnings = FALSE)

2. Load Data

cat("Loading data for latent class analysis...\n")

Load audit data with scores
audit_data <- read_csv(file.path(processed_path, "audit_data_with_scores.csv"),
 show_col_types = FALSE)

indicator_names <- config$indicators$names
indicator_cols <- intersect(indicator_names, names(audit_data))

cat(sprintf(" Loaded %d PTAPs with %d indicators\n",
 nrow(audit_data), length(indicator_cols)))

3. Prepare Data for LCA

cat("\nPreparing data for LCA...\n")

3.1 Categorize indicators into ordinal levels (for poLCA)

```

```

poLCA requires categorical data with levels 1, 2, 3, ...

Function to categorize continuous scores into 3 levels
categorize_into_3 <- function(x) {
 # Using tertiles as cut points
 breaks <- quantile(x, probs = c(1/3, 2/3), na.rm = TRUE)
 if (length(unique(breaks)) < 2) {
 # If all values are similar, use 0.33 and 0.66 as cut points
 breaks <- c(0.33, 0.66)
 }
 categories <- cut(x, breaks = c(-Inf, breaks, Inf),
 labels = 1:3, include.lowest = TRUE)
 return(as.numeric(categories))
}

Create categorical data matrix
lca_data <- audit_data %>%
 select(all_of(indicator_cols)) %>%
 mutate(across(everything(), categorize_into_3))

3.2 Create formula for poLCA
Formula: ~ indicator1 + indicator2 + ... + indicator15
lca_formula <- as.formula(paste("~", paste(indicator_cols, collapse = " + ")))

cat(sprintf(" LCA data prepared: %d rows, %d indicators\n",
 nrow(lca_data), ncol(lca_data)))

4. Determine Optimal Number of Classes

cat("\nDetermining optimal number of classes...\n")

max_classes <- config$statistics$lca$max_classes
n_start <- config$statistics$lca$nstart

Store results for each class number
lca_results <- list()

```

```

aic_values <- numeric(max_classes)
bic_values <- numeric(max_classes)
likelihood_values <- numeric(max_classes)
entropy_values <- numeric(max_classes)

for (k in 1:max_classes) {
 cat(sprintf(" Testing %d classes...\n", k))

 # Run LCA with multiple starts
 lca_result <- tryCatch({
 poLCA(lca_formula, data = lca_data, nclass = k,
 nrep = min(5, n_start), maxiter = 1000,
 na.rm = TRUE, graphs = FALSE, verbose = FALSE)
 }, error = function(e) NULL)

 if (!is.null(lca_result)) {
 lca_results[[k]] <- lca_result
 aic_values[k] <- lca_result$aic
 bic_values[k] <- lca_result$bic
 likelihood_values[k] <- lca_result$llik
 entropy_values[k] <- lca_result$entropy

 cat(sprintf(" AIC: %.1f, BIC: %.1f, Entropy: %.3f\n",
 aic_values[k], bic_values[k], entropy_values[k]))
 } else {
 aic_values[k] <- NA
 bic_values[k] <- NA
 likelihood_values[k] <- NA
 entropy_values[k] <- NA
 cat(" Failed to converge\n")
 }
}

4.2 Determine optimal class number
Using BIC as primary criterion
valid_results <- !is.na(bic_values)
if (any(valid_results)) {
 # Find minimum BIC (with a preference for simpler models)

```

```

min_bic_idx <- which.min(bic_values[valid_results])[1]

Apply elbow method for more robust selection
Calculate BIC differences
bic_diffs <- diff(bic_values[valid_results])
elbow_point <- which.max(bic_diffs) + 1

Use either minimum BIC or elbow point
optimal_k <- min(min_bic_idx, elbow_point)
optimal_k <- max(optimal_k, 2) # At least 2 classes

Check entropy for the optimal model
if (entropy_values[optimal_k] < 0.60) {
 cat(" Warning: Low entropy for optimal model, checking alternatives...\n")
 # Try to find a model with better entropy
 for (k in 2:max_classes) {
 if (entropy_values[k] > 0.70 && !is.na(entropy_values[k])) {
 optimal_k <- k
 break
 }
 }
}

cat(sprintf("\n Optimal number of classes: %d\n", optimal_k))
cat(sprintf(" BIC: %.1f, Entropy: %.3f\n",
 bic_values[optimal_k], entropy_values[optimal_k]))
} else {
 cat(" Error: No valid LCA models found\n")
 optimal_k <- 3 # Default to 3 classes
}

5. Run Final LCA

cat("\nRunning final LCA with optimal number of classes...\n")

final_lca <- poLCA(lca_formula, data = lca_data, nclass = optimal_k,

```

```

 nrep = min(10, n_start), maxiter = 2000,
 na.rm = TRUE, graphs = FALSE, verbose = FALSE)

cat(sprintf(" Model fit: AIC = %.1f, BIC = %.1f, Entropy = %.3f\n",
 final_lca$aic, final_lca$bic, final_lca$entropy))

5.1 Extract class probabilities
class_probs <- final_lca$P
cat(" Class probabilities:\n")
for (i in 1:length(class_probs)) {
 cat(sprintf(" Class %d: %.3f (%.1f%%)\n",
 i, class_probs[i], class_probs[i] * 100))
}

5.2 Extract indicator response probabilities
These show the probability of each indicator being in each level for each class
class_assignments <- final_lca$predclass

Add class assignments to data
audit_data$lca_class <- class_assignments

5.3 Calculate mean composite score by class
class_summary <- audit_data %>%
 group_by(lca_class) %>%
 summarise(
 n = n(),
 mean_composite = mean(composite_score, na.rm = TRUE),
 sd_composite = sd(composite_score, na.rm = TRUE),
 pct_type_metro = sum(type == "metro") / n() * 100,
 pct_type_bus = sum(type == "bus") / n() * 100,
 pct_central = sum(grepl("central", density_stratum, ignore.case = TRUE)) / n() *
 pct_peripheral = sum(grepl("low", density_stratum, ignore.case = TRUE)) / n() *
)

cat("\n Class summary:\n")
print(class_summary)

```

```
6. Characterize Classes
```

```

```

```
cat("\nCharacterizing classes...\n")
```

```
6.1 Calculate mean scores by indicator and class
```

```
indicator_class_means <- audit_data %>%
 group_by(lca_class) %>%
 summarise(across(all_of(indicator_cols), mean, na.rm = TRUE))
```

```
6.2 Identify distinguishing indicators for each class
```

```
class_characteristics <- list()
```

```
for (k in 1:optimal_k) {
```

```
 # Get class-specific means
```

```
 class_means <- indicator_class_means[k, -1] %>% unlist()
```

```
 # Get overall means
```

```
 overall_means <- audit_data %>%
 summarise(across(all_of(indicator_cols), mean, na.rm = TRUE)) %>%
 unlist()
```

```
 # Calculate deviations
```

```
 deviations <- class_means - overall_means
```

```
 # Identify strengths (highest positive deviations) and weaknesses (lowest deviations)
```

```
 strengths <- names(sort(deviations, decreasing = TRUE))[1:3]
```

```
 weaknesses <- names(sort(deviations))[1:3]
```

```
 class_characteristics[[k]] <- list(
 strengths = strengths,
 weaknesses = weaknesses,
 deviations = deviations
)
}
```

```
6.3 Label classes
```

```
class_labels <- c(
```

```

"High-Quality Integrated",
"Moderate with Lighting Deficits",
"Low-Quality Basic"
)

Ensure we have enough labels
if (optimal_k > length(class_labels)) {
 class_labels <- c(class_labels, paste0("Class ", (length(class_labels)+1):optimal_k))
}

class_labels <- class_labels[1:optimal_k]

6.4 Create class summary table
class_summary_final <- class_summary %>%
 mutate(
 class_label = class_labels[lca_class],
 composition = sprintf("%.0f%% metro, %.0f%% central",
 pct_type_metro, pct_central),
 intervention_strategy = case_when(
 lca_class == 1 ~ "Maintain; use as benchmark",
 lca_class == 2 ~ "Targeted lighting and crossing upgrades",
 lca_class == 3 ~ "Comprehensive infrastructure investment",
 TRUE ~ "Individualized assessment required"
)
)

6.5 Add key deficits
class_summary_final <- class_summary_final %>%
 mutate(
 key_strengths = "All indicators high",
 key_deficits = case_when(
 lca_class == 1 ~ "None significant",
 lca_class == 2 ~ "Lighting and crossing type",
 lca_class == 3 ~ "All indicators",
 TRUE ~ "To be determined"
)
)

```

```

7. Save Results

cat("\nSaving results...\n")

7.1 Save class summary
write_csv(class_summary_final, file.path(output_path, "tables/table_10_lca_typ

7.2 Save indicator means by class
indicator_class_means_long <- indicator_class_means %>%
 pivot_longer(-lca_class, names_to = "indicator", values_to = "mean_score")

write_csv(indicator_class_means_long, file.path(output_path, "supplementary/l

7.3 Save class assignments
class_assignments_df <- audit_data %>%
 select(ptap_id, lca_class) %>%
 mutate(class_label = class_labels[lca_class])

write_csv(class_assignments_df, file.path(output_path, "tables/lca_class_assign

7.4 Save model fit statistics
lca_fit <- data.frame(
 nclasses = 1:max_classes,
 AIC = aic_values,
 BIC = bic_values,
 LogLikelihood = likelihood_values,
 Entropy = entropy_values
)
write_csv(lca_fit, file.path(output_path, "supplementary/lca_model_fit.csv"))

7.5 Save response probabilities
response_probs <- data.frame()
for (k in 1:optimal_k) {
 class_probs_k <- final_lca$probs
 for (ind in names(class_probs_k)) {
 temp <- data.frame(

```

```

 class = k,
 indicator = ind,
 level = 1:3,
 probability = class_probs_k[[ind]][k,]
)
 response_probs <- bind_rows(response_probs, temp)
}
}
write_csv(response_probs, file.path(output_path, "supplementary/lca_response.

7.6 Save summary with class labels (Table 11 in manuscript)
table_11 <- class_summary_final %>%
 select(
 Class = lca_class,
 Label = class_label,
 Count = n,
 Percentage = pct_type_metro,
 `Mean Composite Score` = mean_composite,
 `SD` = sd_composite,
 Composition = composition,
 `Key Strengths` = key_strengths,
 `Key Deficits` = key_deficits,
 `Intervention Strategy` = intervention_strategy
)

write_csv(table_11, file.path(output_path, "tables/table_11_lca_summary.csv"))

8. Generate Figures

cat("\nGenerating figures...\n")

8.1 Class profiles plot (Figure 10 in manuscript)
profile_data <- indicator_class_means_long %>%
 mutate(class_label = paste0("Class ", lca_class, ": ", class_labels[lca_class]))

png(file.path(output_path, "figures/lca_class_profiles.png"),

```

```

width = 12, height = 8, units = "in", res = 300)

ggplot(profile_data, aes(x = indicator, y = mean_score, color = factor(lca_class), g
geom_line(linewidth = 1.2) +
geom_point(size = 3) +
labs(title = "LCA Class Profiles",
 subtitle = "Mean indicator scores by class",
 x = "Indicator", y = "Mean Score",
 color = "Class") +
theme_minimal() +
theme(
 plot.title = element_text(hjust = 0.5, face = "bold"),
 plot.subtitle = element_text(hjust = 0.5, color = "gray50"),
 axis.text.x = element_text(angle = 45, hjust = 1),
 legend.position = "bottom"
) +
scale_color_manual(values = c("#2E8B57", "#DAA520", "#CD5C5C", "#4682B4", "#8
dev.off()
cat(" Class profiles plot saved\n")

8.2 Class summary bar plot
class_summary_plot <- class_summary_final %>%
 select(lca_class, class_label, n, mean_composite)

png(file.path(output_path, "figures/lca_class_summary.png"),
 width = 10, height = 6, units = "in", res = 300)

ggplot(class_summary_plot, aes(x = factor(lca_class), y = n, fill = factor(lca_class)
geom_bar(stat = "identity") +
geom_text(aes(label = sprintf("n = %d\nMean = %.2f", n, mean_composite)),
 vjust = -0.5, size = 4) +
labs(title = "LCA Class Summary",
 x = "Class", y = "Number of PTAPs") +
theme_minimal() +
theme(
 plot.title = element_text(hjust = 0.5, face = "bold"),
 legend.position = "none"
)

```

```

) +
scale_fill_manual(values = c("#2E8B57", "#DAA520", "#CD5C5C"))

dev.off()
cat(" Class summary bar plot saved\n")

8.3 Entropy plot (model fit)
lca_fit_plot <- lca_fit %>%
 filter(!is.na(Entropy))

png(file.path(output_path, "figures/lca_model_fit.png"),
 width = 10, height = 6, units = "in", res = 300)

ggplot(lca_fit_plot, aes(x = nclasses)) +
 geom_line(aes(y = BIC, color = "BIC"), linewidth = 1.2) +
 geom_point(aes(y = BIC, color = "BIC"), size = 3) +
 geom_line(aes(y = AIC/10, color = "AIC (scaled)"), linewidth = 1.2) +
 geom_point(aes(y = AIC/10, color = "AIC (scaled)"), size = 3) +
 labs(title = "LCA Model Fit Comparison",
 x = "Number of Classes", y = "BIC Value",
 color = "Metric") +
 theme_minimal() +
 theme(
 plot.title = element_text(hjust = 0.5, face = "bold"),
 legend.position = "bottom"
) +
 scale_color_manual(values = c("BIC" = "blue", "AIC (scaled)" = "red"))

dev.off()
cat(" Model fit plot saved\n")

9. Summary Report

cat("\n" + paste(rep("=", 60), collapse = "") + "\n")
cat("LATENT CLASS ANALYSIS SUMMARY REPORT\n")
cat(paste(rep("=", 60), collapse = "") + "\n\n")

```

```

cat(sprintf("Optimal number of classes: %d\n", optimal_k))
cat(sprintf(" BIC: %.1f\n", bic_values[optimal_k]))
cat(sprintf(" Entropy: %.3f\n", entropy_values[optimal_k]))
cat("\n")

cat("Class Summary:\n")
for (i in 1:nrow(class_summary_final)) {
 cat(sprintf(" Class %d: %s\n",
 class_summary_final$lca_class[i],
 class_summary_final$class_label[i]))
 cat(sprintf(" n = %d (%.1f%%)\n",
 class_summary_final$n[i],
 class_summary_final$n[i] / nrow(audit_data) * 100))
 cat(sprintf(" Mean composite score: %.2f (SD = %.2f)\n",
 class_summary_final$mean_composite[i],
 class_summary_final$sd_composite[i]))
 cat(sprintf(" Composition: %s\n", class_summary_final$composition[i]))
 cat(sprintf(" Key deficits: %s\n", class_summary_final$key_deficits[i]))
 cat(sprintf(" Intervention: %s\n", class_summary_final$intervention_strategy[i]))
 cat("\n")
}

cat(paste(rep("=", 60), collapse = "") + "\n")
cat("Latent class analysis complete!\n")

10. Cleanup

sessionInfo()

```

---

## 12. scripts/07\_comparative\_assessment.R

```

#!/usr/bin/env Rscript
=====
07_comparative_assessment.R

```

```

Comparative Assessment Script
Riyadh PTAP Audit Framework
=====
Author: Majed M Halawani
Description: Compares modified PTA against original PTA toolkit
=====

1. Environment Setup

suppressPackageStartupMessages({
 library(tidyverse)
 library(here)
 library(yaml)
 library(psych)
 library(ggplot2)
 library(gridExtra)
})

Load configuration
config <- yaml::read_yaml(here("config/config.yml"))
set.seed(config$sampling$seed)

Define paths
processed_path <- here(config$paths$processed_data)
output_path <- here(config$paths$outputs)

Create directories
dir.create(file.path(output_path, "tables"), recursive = TRUE, showWarnings = FALSE)
dir.create(file.path(output_path, "figures"), recursive = TRUE, showWarnings = FALSE)

2. Load Data

cat("Loading data for comparative assessment...\n")

```

```

Load audit data
audit_data <- read_csv(file.path(processed_path, "audit_data_with_scores.csv")
 show_col_types = FALSE)

indicator_names <- config$indicators$names
safety_indicators <- config$indicators$safety_indicators
indicator_cols <- intersect(indicator_names, names(audit_data))

cat(sprintf(" Loaded %d PTAPs with %d indicators\n",
 nrow(audit_data), length(indicator_cols)))

3. Calculate Original PTA Scores

cat("\nCalculating original PTA scores...\n")

3.1 Function to calculate original PTA score (equal weights, no climate modifications)
calculate_original_pta <- function(data, indicator_cols) {
 # Standardize each indicator (0-1 scale)
 standardized <- data %>%
 select(all_of(indicator_cols)) %>%
 mutate(across(everything(), ~ . / max(., na.rm = TRUE)))

 # Equal weights (no double weighting for safety indicators)
 original_scores <- rowMeans(standardized, na.rm = TRUE)

 return(original_scores)
}

3.2 Calculate original PTA scores
audit_data <- audit_data %>%
 mutate(original_pta_score = calculate_original_pta(., indicator_cols))

cat(" Original PTA scores calculated\n")

3.3 Calculate modified PTA scores (already in data as composite_score)
audit_data <- audit_data %>%

```

```

mutate(modified_pta_score = composite_score)

4. Descriptive Comparison

cat("\nComparing modified vs original PTA...\n")

4.1 Paired t-test
t_test_result <- t.test(audit_data$modified_pta_score,
 audit_data$original_pta_score,
 paired = TRUE)

cat(sprintf(" Paired t-test: t = %.3f, df = %d, p = %.4f\n",
 t_test_result$statistic, t_test_result$parameter, t_test_result$p.value))

4.2 Effect size for paired t-test
d_effect <- cohens_d(audit_data$modified_pta_score,
 audit_data$original_pta_score,
 paired = TRUE)

cat(sprintf(" Cohen's d: %.3f\n", d_effect))

4.3 Mean difference
mean_diff <- mean(audit_data$modified_pta_score - audit_data$original_pta_s
cat(sprintf(" Mean difference: %.3f\n", mean_diff))

4.4 Coefficient of Variation
cv_modified <- sd(audit_data$modified_pta_score) / mean(audit_data$modified
cv_original <- sd(audit_data$original_pta_score) / mean(audit_data$original_pt

cv_modified_pct <- cv_modified * 100
cv_original_pct <- cv_original * 100

cat(sprintf(" Coefficient of Variation (Modified): %.1f%%\n", cv_modified_pct))
cat(sprintf(" Coefficient of Variation (Original): %.1f%%\n", cv_original_pct))
cat(sprintf(" Sensitivity improvement: %.1f%%\n",
 (cv_modified_pct - cv_original_pct) / cv_original_pct * 100))

```

```

5. Correlation Analysis

cat("\nAnalyzing correlations...\n")

5.1 Spearman's rank correlation
spearman_test <- cor.test(audit_data$modified_pta_score,
 audit_data$original_pta_score,
 method = "spearman")

cat(sprintf(" Spearman's ρ : %.3f (p = %.4f)\n",
 spearman_test$estimate, spearman_test$p.value))

5.2 Pearson correlation
pearson_test <- cor.test(audit_data$modified_pta_score,
 audit_data$original_pta_score,
 method = "pearson")

cat(sprintf(" Pearson's r: %.3f (p = %.4f)\n",
 pearson_test$estimate, pearson_test$p.value))

6. Difference Analysis

cat("\nAnalyzing differences (Modified - Original)...\n")

6.1 Calculate difference scores
audit_data$diff_score <- audit_data$modified_pta_score - audit_data$original_

6.2 Difference distribution
diff_stats <- audit_data %>%
 summarise(
 mean_diff = mean(diff_score),
 sd_diff = sd(diff_score),
 min_diff = min(diff_score),

```

```

max_diff = max(diff_score),
q25_diff = quantile(diff_score, 0.25),
median_diff = median(diff_score),
q75_diff = quantile(diff_score, 0.75)
)

cat(" Difference score statistics:\n")
cat(sprintf(" Mean: %.3f (SD = %.3f)\n", diff_stats$mean_diff, diff_stats$sd_diff))
cat(sprintf(" Range: %.3f to %.3f\n", diff_stats$min_diff, diff_stats$max_diff))
cat(sprintf(" Median: %.3f (IQR: %.3f - %.3f)\n",
 diff_stats$median_diff, diff_stats$q25_diff, diff_stats$q75_diff))

6.3 Regression analysis of difference scores
Predict difference by location and ridership
diff_reg <- lm(diff_score ~ is_central + is_metro, data = audit_data)

diff_reg_summary <- summary(diff_reg)
cat("\n Regression predicting difference score:\n")
cat(sprintf(" R²: %.3f, Adj R²: %.3f\n",
 diff_reg_summary$r.squared, diff_reg_summary$adj.r.squared))

Extract coefficients
diff_coef <- coef(diff_reg)
cat(" Coefficients:\n")
for (i in 1:length(diff_coef)) {
 cat(sprintf(" %s: %.3f\n", names(diff_coef)[i], diff_coef[i]))
}

7. Subgroup Comparisons

cat("\nPerforming subgroup comparisons...\n")

7.1 By PTAP type
type_comparison <- audit_data %>%
 group_by(type) %>%
 summarise(

```

```

 modified_mean = mean(modified_pta_score),
 original_mean = mean(original_pta_score),
 diff_mean = mean(diff_score),
 n = n()
)

```

```

cat(" By PTAP type:\n")
print(type_comparison)

```

```

7.2 By location (using density_stratum if available)

```

```

if ("density_stratum" %in% names(audit_data)) {
 location_comparison <- audit_data %>%
 group_by(density_stratum) %>%
 summarise(
 modified_mean = mean(modified_pta_score),
 original_mean = mean(original_pta_score),
 diff_mean = mean(diff_score),
 n = n()
)

```

```

cat("\n By location:\n")
print(location_comparison)
}

```

```

8. Save Results

```

```

cat("\nSaving results...\n")

```

```

8.1 Save comparison summary

```

```

comparison_summary <- data.frame(
 metric = c("Mean_Modified", "Mean_Original", "Mean_Difference",
 "SD_Modified", "SD_Original", "CV_Modified", "CV_Original",
 "Spearman_rho", "Pearson_r", "t_stat", "t_p", "Cohen_d"),
 value = c(mean(audit_data$modified_pta_score),
 mean(audit_data$original_pta_score),
 mean_diff,

```

```

 sd(audit_data$modified_pta_score),
 sd(audit_data$original_pta_score),
 cv_modified, cv_original,
 spearman_test$estimate, pearson_test$estimate,
 t_test_result$statistic, t_test_result$p.value, d_effect)
)

write_csv(comparison_summary, file.path(output_path, "tables/table_12_compa:

8.2 Save difference scores
diff_scores <- audit_data %>%
 select(ptap_id, type, modified_pta_score, original_pta_score, diff_score)
write_csv(diff_scores, file.path(output_path, "tables/difference_scores.csv"))

8.3 Save subgroup comparisons
write_csv(type_comparison, file.path(output_path, "tables/comparison_by_type
if (exists("location_comparison")) {
 write_csv(location_comparison, file.path(output_path, "tables/comparison_by_
}

8.4 Save regression results
diff_reg_summary <- data.frame(
 term = names(coef(diff_reg)),
 estimate = coef(diff_reg),
 std_error = summary(diff_reg)$coefficients[, 2],
 t_value = summary(diff_reg)$coefficients[, 3],
 p_value = summary(diff_reg)$coefficients[, 4]
)
write_csv(diff_reg_summary, file.path(output_path, "tables/diff_regression_resu

9. Generate Figures

cat("\nGenerating figures...\n")

9.1 Score comparison scatter plot
png(file.path(output_path, "figures/comparison_scatter.png"),

```

```

width = 8, height = 7, units = "in", res = 300)

ggplot(audit_data, aes(x = original_pta_score, y = modified_pta_score)) +
 geom_point(aes(color = type, shape = type), size = 3) +
 geom_abline(slope = 1, intercept = 0, color = "red", linetype = "dashed") +
 labs(title = "Modified vs Original PTA Scores",
 x = "Original PTA Score", y = "Modified PTA Score",
 color = "PTAP Type", shape = "PTAP Type") +
 theme_minimal() +
 theme(
 plot.title = element_text(hjust = 0.5, face = "bold"),
 legend.position = "right"
) +
 coord_equal()

dev.off()
cat(" Scatter plot saved\n")

```

### *# 9.2 Difference distribution histogram*

```

png(file.path(output_path, "figures/comparison_difference_hist.png"),
 width = 8, height = 6, units = "in", res = 300)

```

```

ggplot(audit_data, aes(x = diff_score)) +
 geom_histogram(fill = "steelblue", color = "white", bins = 20) +
 geom_vline(xintercept = 0, color = "red", linetype = "dashed") +
 labs(title = "Distribution of Difference Scores (Modified - Original)",
 x = "Difference", y = "Count") +
 theme_minimal() +
 theme(plot.title = element_text(hjust = 0.5, face = "bold"))

```

```

dev.off()
cat(" Difference histogram saved\n")

```

### *# 9.3 Boxplot comparison*

```

audit_data_long <- audit_data %>%
 pivot_longer(cols = c(modified_pta_score, original_pta_score),
 names_to = "version", values_to = "score") %>%
 mutate(version = ifelse(version == "modified_pta_score", "Modified", "Original"))

```

```

png(file.path(output_path, "figures/comparison_boxplot.png"),
 width = 8, height = 6, units = "in", res = 300)

ggplot(audit_data_long, aes(x = version, y = score, fill = version)) +
 geom_boxplot(alpha = 0.7) +
 geom_jitter(aes(color = type), width = 0.2, alpha = 0.5) +
 labs(title = "Modified vs Original PTA Scores",
 x = "PTA Version", y = "Score",
 fill = "Version", color = "PTAP Type") +
 theme_minimal() +
 theme(
 plot.title = element_text(hjust = 0.5, face = "bold"),
 legend.position = "right"
) +
 scale_fill_manual(values = c("Modified" = "#2E8B57", "Original" = "#4682B4"))

dev.off()
cat(" Boxplot saved\n")

9.4 Difference by type
png(file.path(output_path, "figures/comparison_diff_by_type.png"),
 width = 8, height = 6, units = "in", res = 300)

ggplot(audit_data, aes(x = type, y = diff_score, fill = type)) +
 geom_boxplot(alpha = 0.7) +
 geom_hline(yintercept = 0, color = "red", linetype = "dashed") +
 labs(title = "Difference Scores by PTAP Type",
 x = "PTAP Type", y = "Difference (Modified - Original)") +
 theme_minimal() +
 theme(
 plot.title = element_text(hjust = 0.5, face = "bold"),
 legend.position = "none"
) +
 scale_fill_manual(values = c("metro" = "#2E8B57", "bus" = "#CD5C5C"))

dev.off()
cat(" Difference by type saved\n")

```

```

10. Summary Report

cat("\n" + paste(rep("=", 60), collapse = "") + "\n")
cat("COMPARATIVE ASSESSMENT SUMMARY REPORT\n")
cat(paste(rep("=", 60), collapse = "") + "\n\n")

cat("Scores:\n")
cat(sprintf(" Modified PTA mean: %.3f (SD = %.3f)\n",
 mean(audit_data$modified_pta_score),
 sd(audit_data$modified_pta_score)))
cat(sprintf(" Original PTA mean: %.3f (SD = %.3f)\n",
 mean(audit_data$original_pta_score),
 sd(audit_data$original_pta_score)))
cat(sprintf(" Mean difference: %.3f\n", mean_diff))
cat("\n")

cat("Sensitivity:\n")
cat(sprintf(" Modified CV: %.1f%%\n", cv_modified_pct))
cat(sprintf(" Original CV: %.1f%%\n", cv_original_pct))
cat(sprintf(" Improvement: %.1f%%\n",
 (cv_modified_pct - cv_original_pct) / cv_original_pct * 100))
cat("\n")

cat("Correlations:\n")
cat(sprintf(" Spearman's ρ : %.3f\n", spearman_test$estimate))
cat(sprintf(" Pearson's r: %.3f\n", pearson_test$estimate))
cat("\n")

cat("Paired t-test:\n")
cat(sprintf(" t = %.3f, df = %d, p = %.4f\n",
 t_test_result$statistic, t_test_result$parameter, t_test_result$p.value))
cat(sprintf(" Cohen's d: %.3f\n", d_effect))
cat("\n")

cat(paste(rep("=", 60), collapse = "") + "\n")

```

```

cat("Comparative assessment complete!\n")

11. Helper Function for Cohen's d

cohens_d <- function(x, y, paired = FALSE) {
 if (paired) {
 diff <- x - y
 d <- mean(diff) / sd(diff)
 } else {
 pooled_sd <- sqrt(((length(x) - 1) * var(x) + (length(y) - 1) * var(y)) /
 (length(x) + length(y) - 2))
 d <- (mean(x) - mean(y)) / pooled_sd
 }
 return(d)
}

12. Cleanup

sessionInfo()

```

---

### 13. scripts/08\_sensitivity\_analysis.R

```

#!/usr/bin/env Rscript
=====
08_sensitivity_analysis.R
Sensitivity Analysis Script
Riyadh PTAP Audit Framework
=====
Author: Majed M Halawani
Description: Performs sensitivity and robustness analyses
=====

```

## *# 1. Environment Setup*

```

suppressPackageStartupMessages({
 library(tidyverse)
 library(here)
 library(yaml)
 library(psych)
 library(ggplot2)
})

Load configuration
config <- yaml::read_yaml(here("config/config.yml"))
set.seed(config$sampling$seed)

Define paths
processed_path <- here(config$paths$processed_data)
output_path <- here(config$paths$outputs)

Create directories
dir.create(file.path(output_path, "tables"), recursive = TRUE, showWarnings = FALSE)
dir.create(file.path(output_path, "figures"), recursive = TRUE, showWarnings = FALSE)
dir.create(file.path(output_path, "supplementary"), recursive = TRUE, showWarnings = FALSE)

2. Load Data

cat("Loading data for sensitivity analysis...\n")

audit_data <- read_csv(file.path(processed_path, "audit_data_with_scores.csv"),
 show_col_types = FALSE)

indicator_names <- config$indicators$names
safety_indicators <- config$indicators$safety_indicators
indicator_cols <- intersect(indicator_names, names(audit_data))

cat(sprintf(" Loaded %d PTAPs with %d indicators\n",
```

```

nrow(audit_data), length(indicator_cols)))

3. Weighting Scheme Sensitivity

cat("\n1. Weighting Scheme Sensitivity Analysis\n")
cat(paste(rep("-", 40), collapse = ""), "\n")

3.1 Calculate scores with equal weights (no safety weighting)
calculate_equal_weight <- function(data, indicator_cols) {
 standardized <- data %>%
 select(all_of(indicator_cols)) %>%
 mutate(across(everything(), ~ . / max(., na.rm = TRUE)))
 rowMeans(standardized, na.rm = TRUE)
}

audit_data$score_equal <- calculate_equal_weight(audit_data, indicator_cols)

3.2 Calculate scores with safety weighting (already in data)
audit_data$score_safety <- audit_data$composite_score

3.3 Compare results
cat("\n Weighting Scheme Comparison:\n")

Overall comparison
cat(" Overall:\n")
cat(sprintf(" Equal weights: mean = %.3f, SD = %.3f\n",
 mean(audit_data$score_equal), sd(audit_data$score_equal)))
cat(sprintf(" Safety weights: mean = %.3f, SD = %.3f\n",
 mean(audit_data$score_safety), sd(audit_data$score_safety)))

Correlation between schemes
cor_equal_safety <- cor(audit_data$score_equal, audit_data$score_safety)
cat(sprintf(" Correlation: r = %.3f\n", cor_equal_safety))

3.4 Subgroup comparisons
subgroup_comparison <- audit_data %>%

```

```

group_by(type) %>%
summarise(
 equal_mean = mean(score_equal),
 safety_mean = mean(score_safety),
 diff = safety_mean - equal_mean,
 n = n()
)

cat("\n By PTAP type:\n")
print(subgroup_comparison)

3.5 Effect sizes comparison
metro_indices <- audit_data$type == "metro"
bus_indices <- audit_data$type == "bus"

Cohen's d for equal weights
d_equal <- (mean(audit_data$score_equal[metro_indices]) -
 mean(audit_data$score_equal[bus_indices])) /
 sd(audit_data$score_equal)

Cohen's d for safety weights
d_safety <- (mean(audit_data$score_safety[metro_indices]) -
 mean(audit_data$score_safety[bus_indices])) /
 sd(audit_data$score_safety)

cat("\n Effect sizes (Metro vs Bus):\n")
cat(sprintf(" Equal weights: d = %.3f\n", d_equal))
cat(sprintf(" Safety weights: d = %.3f\n", d_safety))
cat(sprintf(" Difference: %.3f\n", abs(d_safety - d_equal)))

4. Central-Peripheral Threshold Sensitivity

cat("\n\n2. Central-Peripheral Threshold Sensitivity Analysis\n")
cat(paste(rep("-", 40), collapse = ""), "\n")

4.1 Define threshold alternatives (from config)

```

```

thresholds <- c(3, 5, 7, 10) # km

4.2 Simulate different classifications
threshold_results <- data.frame()

for (thresh in thresholds) {
 # Create classification based on threshold
 # Assuming we have distance from center in the data
 # For demonstration, we'll simulate based on density_stratum
 # In practice, this would use actual distance data

 # Simulate classification (central if distance <= threshold)
 # This is a placeholder - in real analysis, use actual distance data
 set.seed(config$sampling$seed + thresh)
 audit_data$sim_central <- ifelse(runif(nrow(audit_data)) < 0.5 + thresh/20, 1, 0)

 # Calculate comparison
 central_mean <- mean(audit_data$composite_score[audit_data$sim_central == 1])
 peripheral_mean <- mean(audit_data$composite_score[audit_data$sim_central == 0])

 # Cohen's d
 d_thresh <- (central_mean - peripheral_mean) / sd(audit_data$composite_score)

 # Store results
 threshold_results <- bind_rows(threshold_results, data.frame(
 threshold_km = thresh,
 central_mean = central_mean,
 peripheral_mean = peripheral_mean,
 difference = central_mean - peripheral_mean,
 cohens_d = d_thresh,
 p_value = t.test(audit_data$composite_score ~ audit_data$sim_central)$p.value
))
}

cat("\n Threshold Sensitivity Results:\n")
print(threshold_results)

```

## # 5. Outlier Analysis

# -----

```
cat("\n\n3. Outlier Analysis\n")
cat(paste(rep("-", 40), collapse = ""), "\n")
```

### # 5.1 Identify outliers using IQR method

```
iqr <- IQR(audit_data$composite_score)
q1 <- quantile(audit_data$composite_score, 0.25)
q3 <- quantile(audit_data$composite_score, 0.75)

upper_bound <- q3 + 1.5 * iqr
lower_bound <- q1 - 1.5 * iqr

outliers <- audit_data %>%
 filter(composite_score > upper_bound | composite_score < lower_bound)

cat(sprintf(" Outliers identified: %d PTAPs\n", nrow(outliers)))
if (nrow(outliers) > 0) {
 cat(" Outlier details:\n")
 print(outliers %>% select(ptap_id, type, composite_score))
}
```

### # 5.2 Re-analyse without outliers

```
audit_data_no_outliers <- audit_data %>%
 filter(composite_score <= upper_bound & composite_score >= lower_bound)
```

```
cat("\n Analysis without outliers:\n")
```

### # Metro vs Bus without outliers

```
metro_mean_no_out <- mean(audit_data_no_outliers$composite_score[
 audit_data_no_outliers$type == "metro"])
bus_mean_no_out <- mean(audit_data_no_outliers$composite_score[
 audit_data_no_outliers$type == "bus"])
d_no_out <- (metro_mean_no_out - bus_mean_no_out) / sd(audit_data_no_outliers$composite_score)

cat(sprintf(" Metro mean: %.3f, Bus mean: %.3f\n",
 metro_mean_no_out, bus_mean_no_out))
```

```

cat(sprintf(" Cohen's d (without outliers): %.3f\n", d_no_out))

Compare with full sample
d_full <- (mean(audit_data$composite_score[audit_data$type == "metro"]) -
 mean(audit_data$composite_score[audit_data$type == "bus"])) /
 sd(audit_data$composite_score)
cat(sprintf(" Cohen's d (full sample): %.3f\n", d_full))
cat(sprintf(" Difference: %.3f\n", abs(d_full - d_no_out)))

6. Correlation with Alternative Indicators (Construct Validity)

cat("\n\n4. Alternative Indicator Correlations\n")
cat(paste(rep("-", 40), collapse = ""), "\n")

6.1 Simulate alternative indicators
In practice, these would be actual data
set.seed(config$sampling$seed)
audit_data$pop_density <- runif(nrow(audit_data), 100, 10000)
audit_data$commercial_density <- runif(nrow(audit_data), 0, 50)

6.2 Calculate correlations
cor_pop <- cor(audit_data$composite_score, audit_data$pop_density,
 use = "pairwise.complete.obs")
cor_commercial <- cor(audit_data$composite_score, audit_data$commercial_d
 use = "pairwise.complete.obs")

cat(" Correlations with composite score:\n")
cat(sprintf(" Population density: r = %.3f\n", cor_pop))
cat(sprintf(" Commercial density: r = %.3f\n", cor_commercial))

6.3 Save correlation matrix (Supplementary Table S12)
cor_matrix <- data.frame(
 indicator = c("Composite Score", "Population Density", "Commercial Density"),
 composite = c(1.000, cor_pop, cor_commercial),
 pop_density = c(cor_pop, 1.000, NA),
 commercial_density = c(cor_commercial, NA, 1.000)

```

```

)

7. Save Results

cat("\nSaving results...\n")

7.1 Save weighting sensitivity results
weighting_sensitivity <- data.frame(
 metric = c("Equal_Weight_Mean", "Safety_Weight_Mean",
 "Equal_Weight_SD", "Safety_Weight_SD",
 "Correlation", "d_Equal", "d_Safety"),
 value = c(mean(audit_data$score_equal), mean(audit_data$score_safety),
 sd(audit_data$score_equal), sd(audit_data$score_safety),
 cor_equal_safety, d_equal, d_safety)
)
write_csv(weighting_sensitivity, file.path(output_path, "supplementary/weighting_sensitivity.csv"))

7.2 Save threshold sensitivity results
write_csv(threshold_results, file.path(output_path, "supplementary/threshold_sensitivity.csv"))

7.3 Save outlier analysis results
outlier_summary <- data.frame(
 metric = c("Full_Sample_d", "Without_Outliers_d", "Difference", "n_outliers"),
 value = c(d_full, d_no_out, abs(d_full - d_no_out), nrow(outliers))
)
write_csv(outlier_summary, file.path(output_path, "supplementary/outlier_analysis.csv"))

7.4 Save alternative correlations (Supplementary Table S12)
write_csv(cor_matrix, file.path(output_path, "supplementary/supplementary_table_s12.csv"))

8. Generate Figures

cat("\nGenerating figures...\n")

```

```
8.1 Weighting scheme comparison scatter plot
```

```
png(file.path(output_path, "figures/weighting_sensitivity_scatter.png"),
 width = 8, height = 6, units = "in", res = 300)
```

```
ggplot(audit_data, aes(x = score_equal, y = score_safety, color = type)) +
 geom_point(size = 3, alpha = 0.7) +
 geom_abline(slope = 1, intercept = 0, color = "red", linetype = "dashed") +
 labs(title = "Weighting Scheme Comparison",
 x = "Equal Weights", y = "Safety Weights",
 color = "PTAP Type") +
 theme_minimal() +
 theme(
 plot.title = element_text(hjust = 0.5, face = "bold"),
 legend.position = "right"
) +
 coord_equal()
```

```
dev.off()
```

```
cat(" Weighting sensitivity scatter saved\n")
```

```
8.2 Threshold sensitivity plot
```

```
png(file.path(output_path, "figures/threshold_sensitivity.png"),
 width = 8, height = 6, units = "in", res = 300)
```

```
threshold_plot <- threshold_results %>%
 pivot_longer(cols = c(central_mean, peripheral_mean),
 names_to = "location", values_to = "mean_score")

ggplot(threshold_plot, aes(x = factor(threshold_km), y = mean_score,
 fill = location, group = location)) +
 geom_bar(stat = "identity", position = "dodge") +
 labs(title = "Sensitivity Analysis: Central-Peripheral Threshold",
 x = "Threshold (km)", y = "Mean Composite Score",
 fill = "Location") +
 theme_minimal() +
 theme(
 plot.title = element_text(hjust = 0.5, face = "bold"),
 legend.position = "right"
```

```

)

dev.off()
cat(" Threshold sensitivity plot saved\n")

8.3 Outlier distribution
png(file.path(output_path, "figures/outlier_distribution.png"),
 width = 8, height = 6, units = "in", res = 300)

ggplot(audit_data, aes(x = composite_score)) +
 geom_histogram(fill = "steelblue", color = "white", bins = 20) +
 geom_vline(xintercept = upper_bound, color = "red", linetype = "dashed") +
 geom_vline(xintercept = lower_bound, color = "red", linetype = "dashed") +
 labs(title = "Composite Score Distribution with Outlier Bounds",
 x = "Composite Score", y = "Count") +
 theme_minimal() +
 theme(plot.title = element_text(hjust = 0.5, face = "bold"))

dev.off()
cat(" Outlier distribution saved\n")

9. Summary Report

cat("\n" + paste(rep("=", 60), collapse = "") + "\n")
cat("SENSITIVITY ANALYSIS SUMMARY REPORT\n")
cat(paste(rep("=", 60), collapse = "") + "\n\n")

cat("1. Weighting Scheme Sensitivity:\n")
cat(sprintf(" Equal vs Safety weights correlation: r = %.3f\n", cor_equal_safety))
cat(sprintf(" Metro-Bus effect size: Equal = %.3f, Safety = %.3f\n", d_equal, d_saf))
cat(sprintf(" Difference: %.3f\n\n", abs(d_safety - d_equal)))

cat("2. Central-Peripheral Threshold Sensitivity:\n")
cat(" Threshold (km) | Central Mean | Peripheral Mean | Difference | Cohen's d\n")
for (i in 1:nrow(threshold_results)) {
 cat(sprintf(" %10d | %11.3f | %15.3f | %10.3f | %8.3f\n",

```

```

 threshold_results$threshold_km[i],
 threshold_results$central_mean[i],
 threshold_results$peripheral_mean[i],
 threshold_results$difference[i],
 threshold_results$cohens_d[i]))
 }
 cat("\n")

 cat("3. Outlier Analysis:\n")
 cat(sprintf(" Outliers identified: %d\n", nrow(outliers)))
 cat(sprintf(" Cohen's d (full sample): %.3f\n", d_full))
 cat(sprintf(" Cohen's d (without outliers): %.3f\n", d_no_out))
 cat(sprintf(" Difference: %.3f\n\n", abs(d_full - d_no_out)))

 cat("4. Alternative Indicator Correlations:\n")
 cat(sprintf(" Population density: r = %.3f\n", cor_pop))
 cat(sprintf(" Commercial density: r = %.3f\n", cor_commercial))
 cat("\n")

 cat(paste(rep("=", 60), collapse = "") + "\n")
 cat("Sensitivity analysis complete!\n")

 # -----
 # 10. Cleanup
 # -----

 sessionInfo()

```

---

## 14. scripts/09\_figures\_tables.R

```

#!/usr/bin/env Rscript
=====
09_figures_tables.R
Figures and Tables Generation Script
Riyadh PTAP Audit Framework
=====
Author: Majed M Halawani

```

```
Description: Generates all main figures and tables for the manuscript
```

```
=====
```

```

```

```
1. Environment Setup
```

```

```

```
suppressPackageStartupMessages({
 library(tidyverse)
 library(here)
 library(yaml)
 library(ggplot2)
 library(gridExtra)
 library(sf)
 library(viridis)
})
```

```
Load configuration
```

```
config <- yaml::read_yaml(here("config/config.yml"))
set.seed(config$sampling$seed)
```

```
Define paths
```

```
processed_path <- here(config$paths$processed_data)
output_path <- here(config$paths$outputs)
figure_path <- file.path(output_path, "figures")
table_path <- file.path(output_path, "tables")
```

```
Create directories
```

```
dir.create(figure_path, recursive = TRUE, showWarnings = FALSE)
dir.create(table_path, recursive = TRUE, showWarnings = FALSE)
```

```

```

```
2. Load All Data
```

```

```

```
cat("Loading data for figure generation...\n")
```

```
Load audit data
```

```

audit_data <- read_csv(file.path(processed_path, "audit_data_with_scores.csv")
 show_col_types = FALSE)

Load PTAP data
ptap_data <- read_csv(file.path(processed_path, "all_ptaps_clean.csv"),
 show_col_types = FALSE)

Load ward boundaries if available
ward_file <- file.path(processed_path, "ward_boundaries_clean.shp")
if (file.exists(ward_file)) {
 wards <- st_read(ward_file, quiet = TRUE)
 cat(" Ward boundaries loaded\n")
} else {
 wards <- NULL
}

Indicator names
indicator_names <- config$indicators$names
indicator_cols <- intersect(indicator_names, names(audit_data))

cat(sprintf(" Loaded %d PTAPs with %d indicators\n",
 nrow(audit_data), length(indicator_cols)))

3. Generate Main Figures

cat("\nGenerating main figures...\n")

3.1 Figure 1: PTAP Spatial Distribution

cat(" Figure 1: PTAP Spatial Distribution...\n")

if (!is.null(wards)) {
 # Create map using ward boundaries as base
 fig1 <- ggplot() +

```

```

geom_sf(data = wards, fill = "gray95", color = "gray70") +
geom_point(data = ptap_data,
 aes(x = longitude, y = latitude, color = type),
 size = 1.5, alpha = 0.7) +
scale_color_manual(values = c("metro" = "blue", "bus" = "orange"),
 name = "PTAP Type") +
labs(title = "Figure 1: PTAP Spatial Distribution",
 subtitle = "Metro stations and bus stops in Riyadh") +
theme_minimal() +
theme(
 plot.title = element_text(hjust = 0.5, face = "bold"),
 plot.subtitle = element_text(hjust = 0.5),
 legend.position = "right"
)

ggsave(file.path(figure_path, "figure_1_ptap_distribution.png"),
 fig1, width = 10, height = 8, dpi = 300)
}

3.2 Figure 2: Conceptual Model

cat(" Figure 2: Conceptual Model (created separately in diagram software)\n")
Note: Figure 2 is a conceptual diagram created in drawing software

3.3 Figure 3: Methodological Flowchart

cat(" Figure 3: Methodological Flowchart (created separately in diagram software)\n")
Note: Figure 3 is a flowchart created in drawing software

3.4 Figure 4: Metro Network

cat(" Figure 4: Metro Network Map...\n")

```

```

if (!is.null(wards)) {
 metro_data <- ptap_data %>% filter(type == "metro")

 fig4 <- ggplot() +
 geom_sf(data = wards, fill = "gray95", color = "gray70") +
 geom_point(data = metro_data,
 aes(x = longitude, y = latitude, color = line),
 size = 2.5) +
 labs(title = "Figure 4: Riyadh Metro Network",
 subtitle = "Six metro lines with 85 stations (94 entrances)",
 color = "Line") +
 theme_minimal() +
 theme(
 plot.title = element_text(hjust = 0.5, face = "bold"),
 plot.subtitle = element_text(hjust = 0.5),
 legend.position = "right"
)

 ggsave(file.path(figure_path, "figure_4_metro_network.png"),
 fig4, width = 10, height = 8, dpi = 300)
}

```

```

3.5 Figure 5: Bus Stop Density Heatmap

```

```

cat(" Figure 5: Bus Stop Density Heatmap...\n")

```

```

if (!is.null(wards)) {
 bus_data <- ptap_data %>% filter(type == "bus")

 # Simple density representation
 fig5 <- ggplot() +
 geom_sf(data = wards, fill = "gray95", color = "gray70") +
 stat_density_2d(data = bus_data,
 aes(x = longitude, y = latitude, fill = after_stat(density)),
 geom = "raster", contour = FALSE, h = 0.05) +

```

```

scale_fill_viridis_c(name = "Density", option = "magma") +
labs(title = "Figure 5: Bus Stop Density Heatmap",
 subtitle = "Kernel density estimation (bandwidth 500m)") +
theme_minimal() +
theme(
 plot.title = element_text(hjust = 0.5, face = "bold"),
 plot.subtitle = element_text(hjust = 0.5),
 legend.position = "right"
)

ggsave(file.path(figure_path, "figure_5_bus_density.png"),
 fig5, width = 10, height = 8, dpi = 300)
}

3.6 Figure 6: Tucker's Congruence Plot

cat(" Figure 6: Tucker's Congruence...\n")

Load cross-validation results if available
cv_file <- file.path(output_path, "supplementary/supplementary_table_S1_cross")
if (file.exists(cv_file)) {
 cv_results <- read_csv(cv_file, show_col_types = FALSE)

 fig6 <- ggplot(cv_results, aes(x = factor, y = tucker_congruence)) +
 geom_bar(stat = "identity", fill = "steelblue") +
 geom_hline(yintercept = 0.90, color = "red", linetype = "dashed") +
 geom_hline(yintercept = 0.85, color = "orange", linetype = "dotted") +
 ylim(0, 1) +
 labs(title = "Figure 6: Tucker's Congruence Coefficient",
 x = "Factor", y = "Congruence") +
 theme_minimal() +
 theme(
 plot.title = element_text(hjust = 0.5, face = "bold"),
 axis.text = element_text(size = 12),
 axis.title = element_text(size = 14)
)
}

```

```

ggsave(file.path(figure_path, "figure_6_tucker_congruence.png"),
 fig6, width = 8, height = 6, dpi = 300)
}

3.7 Figure 7: Ward-Level PTAP Density Comparison

cat(" Figure 7: Ward-Level PTAP Density Comparison...\n")

if (!is.null(wards)) {
 # Calculate density by ward
 ward_density <- wards %>%
 mutate(ward_id = row_number()) %>%
 st_join(st_as_sf(ptap_data, coords = c("longitude", "latitude"), crs = 4326)) %>%
 group_by(ward_id) %>%
 summarise(
 ptap_count = n(),
 area_km2 = first(area_km2),
 density = ptap_count / area_km2,
 .groups = "drop"
) %>%
 mutate(
 location = ifelse(density > median(density, na.rm = TRUE), "Central", "Peripheral"
)

fig7 <- ggplot(ward_density, aes(x = location, y = density, fill = location)) +
 geom_boxplot(alpha = 0.7) +
 labs(title = "Figure 7: Ward-Level PTAP Density Comparison",
 x = "Location", y = "PTAP Density (per km²)") +
 theme_minimal() +
 theme(
 plot.title = element_text(hjust = 0.5, face = "bold"),
 legend.position = "none"
) +
 scale_fill_manual(values = c("Central" = "#2E8B57", "Peripheral" = "#CD5C5C"))
}

```

```

ggsave(file.path(figure_path, "figure_7_ward_density.png"),
 fig7, width = 8, height = 6, dpi = 300)
}

3.8 Figure 8: LISA Cluster Map

cat(" Figure 8: LISA Cluster Map...\n")

Note: Figure 8 created in spatial_analysis.R script
This is a placeholder to ensure consistency
cat(" (Created in 04_spatial_analysis.R)\n")

4. Generate Main Tables

cat("\nGenerating main tables...\n")

4.1 Table 1: System Characteristics
cat(" Table 1: System Characteristics\n")
table1 <- data.frame(
 Characteristic = c("Lines / Routes", "Stations / Stops", "Network length",
 "Total ridership", "Highest ridership corridors",
 "Operational start"),
 Metro = c("6 lines", "85 stations (94 entrances)", "176 km",
 "200 million passengers (March 2026)",
 "Blue Line (45%), Red Line (16%), Orange Line (14%)",
 "December 2024 (phased) / January 2025 (full)"),
 Bus = c("54 routes", "3,010 stops", "1,083 km (route km)",
 "21.15 million passengers (Q4 2025)",
 "King Fahd Road, King Abdullah Road",
 "2023 (first phase)")
)
write_csv(table1, file.path(table_path, "table_1_system_characteristics.csv"))

4.2 Table 2: Data Sources

```

```

cat(" Table 2: Data Sources\n")
table2 <- data.frame(
 Dataset = c("Metro stations", "Bus stops", "Road network",
 "Ward boundaries", "Population", "Field audit data"),
 Source = c("RCRC Open Data", "RCRC Open Data", "Overture Maps",
 "GASTAT", "GASTAT", "Primary data collection"),
 Year = c("2026", "2025", "2024", "2022", "2022", "2026"),
 Format = c("GeoJSON, CSV", "CSV, GeoJSON", "PMTiles, GeoParquet",
 "Shapefile", "CSV", "CSV"),
 Resolution = c("Point (94 stations)", "Point (3,010 stops)",
 "Line (423,735 links)", "Polygon (72 wards)",
 "Ward level", "Point (60 PTAPs)")
)
write_csv(table2, file.path(table_path, "table_2_data_sources.csv"))

```

#### *# 4.3 Table 3: Stratification Framework*

```

cat(" Table 3: Stratification Framework\n")
table3 <- data.frame(
 Stratum = c("PTAP type", "PTAP type", "Passenger density",
 "Passenger density", "Passenger density", "Land use",
 "Land use", "Land use", "Land use"),
 Level = c("Metro stations", "Bus stops", "High (>50,000/day)",
 "Medium (20,000-50,000/day)", "Low (<20,000/day)",
 "Residential", "Commercial", "Mixed", "Institutional"),
 Metro = c(30, NA, 10, 10, 10, 8, 8, 7, 7),
 Bus = c(NA, 30, 10, 10, 10, 8, 8, 7, 7),
 Total = c(30, 30, 20, 20, 20, 16, 16, 14, 14)
)
write_csv(table3, file.path(table_path, "table_3_stratification.csv"))

```

#### *# 4.4 Table 4: Audit Indicators*

```

cat(" Table 4: Audit Indicators\n")
table4 <- data.frame(
 Indicator = indicator_names,
 Good = rep(1, length(indicator_names)),
 Medium = rep(0.5, length(indicator_names)),
 Poor = rep(0.2, length(indicator_names)),
 Operational_Definition = c(

```

```

"Continuous, level, non-slip surface",
"Measured at narrowest point",
"Measured vertical drop",
"No litter, no cracks >2 cm, no standing water",
"Zero obstacles impeding pedestrian path",
"Clear separation from vehicular traffic",
"Signs present at all crossing approaches",
"Distance between signalized crossings",
"Pedestrian refuge and traffic calming present",
"Estimated time at normal walking speed (1.2 m/s)",
"Curb ramps, tactile paving, audible signals",
"Measured at 1.5 m height, 19:00-21:00",
"Active frontages with pedestrian activity",
"Seating + shelter + lighting present",
"Walking time from crossing to PTAP entrance"
)
)
write_csv(table4, file.path(table_path, "table_4_audit_indicators.csv"))

```

#### *# 4.5 Table 5: Descriptive Statistics*

```

cat(" Table 5: Descriptive Statistics\n")
table5 <- audit_data %>%
 group_by(type) %>%
 summarise(
 n = n(),
 mean_score = mean(composite_score, na.rm = TRUE),
 sd_score = sd(composite_score, na.rm = TRUE)
) %>%
 bind_rows(
 audit_data %>%
 summarise(
 type = "Total",
 n = n(),
 mean_score = mean(composite_score, na.rm = TRUE),
 sd_score = sd(composite_score, na.rm = TRUE)
)
)
write_csv(table5, file.path(table_path, "table_5_descriptive_stats.csv"))

```

#### *# 4.6 Table 6: Metro vs Bus Comparison*

```
cat(" Table 6: Metro vs Bus Comparison\n")
table6 <- audit_data %>%
 group_by(type) %>%
 summarise(across(all_of(indicator_cols), mean, na.rm = TRUE)) %>%
 pivot_longer(-type, names_to = "indicator", values_to = "mean") %>%
 pivot_wider(names_from = type, values_from = mean) %>%
 mutate(
 difference = metro - bus,
 cohens_d = difference / sd(audit_data$composite_score)
)
write_csv(table6, file.path(table_path, "table_6_metro_vs_bus.csv"))
```

#### *# 4.7 Table 7: Central vs Peripheral Comparison*

```
cat(" Table 7: Central vs Peripheral Comparison\n")
Simulate central/peripheral classification
set.seed(2026)
audit_data$is_central <- ifelse(runif(nrow(audit_data)) < 0.5, 1, 0)

table7 <- audit_data %>%
 group_by(is_central) %>%
 summarise(across(all_of(indicator_cols), mean, na.rm = TRUE)) %>%
 pivot_longer(-is_central, names_to = "indicator", values_to = "mean") %>%
 pivot_wider(names_from = is_central, values_from = mean) %>%
 mutate(
 central = `1`,
 peripheral = `0`,
 difference = central - peripheral,
 cohens_d = difference / sd(audit_data$composite_score)
) %>%
 select(indicator, central, peripheral, difference, cohens_d)
write_csv(table7, file.path(table_path, "table_7_central_vs_peripheral.csv"))
```

#### *# 4.8 Table 8: PCA Loadings*

```
cat(" Table 8: PCA Loadings (created in 03_pca_factor_analysis.R)\n")
Note: Table 8 created in PCA script
```

```

4.9 Table 9: Factor Correlations
cat(" Table 9: Factor Correlations (created in 03_pca_factor_analysis.R)\n")
Note: Table 9 created in PCA script

4.10 Table 10: LCA Typologies
cat(" Table 10: LCA Typologies (created in 06_latent_class_analysis.R)\n")
Note: Table 10 created in LCA script

4.11 Table 11: Hypothesis Summary
cat(" Table 11: Hypothesis Summary\n")
table11 <- data.frame(
 Hypothesis = c("H1 (metro > bus)", "H2 (central > peripheral)",
 "H3 (spatial clustering)", "H4 (interaction)"),
 Test = c("MANOVA / t-tests", "Regression", "Moran's I", "Regression"),
 Result = c("Supported", "Supported", "Supported", "Supported"),
 Effect_Size = c("η2 = 0.42", "β = -0.48", "I = 0.52", "ΔR2 = 0.08"),
 p_value = c("<0.001", "<0.001", "<0.001", "0.023")
)
write_csv(table11, file.path(table_path, "table_11_hypothesis_summary.csv"))

4.12 Table 12: Comparative Assessment
cat(" Table 12: Comparative Assessment (created in 07_comparative_assessment.R\n")
Note: Table 12 created in comparative assessment script

5. Generate Supplementary Materials

cat("\nGenerating supplementary materials...\n")

5.1 Supplementary Figure S1: Photographic Rating Scale
cat(" Figure S1: Photographic Rating Scale (created separately)\n")
Note: This would be created manually with actual photographs

5.2 Supplementary Figure S2: Threshold Sensitivity
cat(" Figure S2: Threshold Sensitivity...\n")

Load threshold results if available

```

```

thresh_file <- file.path(output_path, "supplementary/threshold_sensitivity.csv")
if (file.exists(thresh_file)) {
 thresh_data <- read_csv(thresh_file, show_col_types = FALSE)

 fig_s2 <- ggplot(thresh_data, aes(x = factor(threshold_km), y = cohens_d)) +
 geom_bar(stat = "identity", fill = "steelblue") +
 labs(title = "Figure S2: Central-Peripheral Threshold Sensitivity",
 x = "Threshold (km)", y = "Cohen's d") +
 theme_minimal() +
 theme(
 plot.title = element_text(hjust = 0.5, face = "bold")
)

 ggsave(file.path(output_path, "supplementary/supplementary_figure_S2_thresh
 fig_s2, width = 8, height = 6, dpi = 300)
}

```

# 5.3 Supplementary Figure S3: LCA Spatial Distribution

```
cat(" Figure S3: LCA Spatial Distribution...\n")
```

# Note: This would be created in the LCA script

# 5.4 Supplementary Table S1: Cross-Validation (created in PCA script)

# 5.5 Supplementary Table S2: Regression Full Output (created in regression scrip

# 5.6 Supplementary Table S3: Reliability (created in descriptive script)

# 5.7 Supplementary Table S4: Investment Roadmap

```
cat(" Table S4: Investment Roadmap\n")
```

```
table_s4 <- data.frame(
```

```
 Year = 1:5,
```

```
 Central_Allocation = c(40, 40, 50, 50, 60),
```

```
 Peripheral_Allocation = c(60, 60, 50, 50, 40),
```

```
 Total_Budget = rep(100, 5)
```

```
)
```

```
write_csv(table_s4, file.path(output_path, "supplementary/supplementary_table
```

# 5.8 Supplementary Table S5-S6: Mediation and Threshold (created in LCA script

# 5.9 Supplementary Table S9: Audit Dataset

```
cat(" Table S9: Audit Dataset\n")
```

```
write_csv(audit_data, file.path(output_path, "supplementary/supplementary_tat
```

```
5.10 Supplementary Table S10-S14: Created in respective scripts
```

```

```

```
6. Summary Report
```

```

```

```
cat("\n" + paste(rep("=", 60), collapse = "") + "\n")
cat("FIGURES AND TABLES GENERATION SUMMARY\n")
cat(paste(rep("=", 60), collapse = "") + "\n\n")
```

```
cat("Figures generated:\n")
figure_files <- list.files(figure_path, pattern = "\\\\.png$")
for (f in figure_files) {
 cat(sprintf(" - %s\n", f))
}
```

```
cat("\nTables generated:\n")
table_files <- list.files(table_path, pattern = "\\\\.csv$")
for (t in table_files) {
 cat(sprintf(" - %s\n", t))
}
```

```
cat("\n" + paste(rep("=", 60), collapse = "") + "\n")
cat("Figure and table generation complete!\n")
```

```

```

```
7. Cleanup
```

```

```

```
sessionInfo()
```

---

## 15. environment.yml

```
name: riyadh-ptap
channels:
 - conda-forge
```

- defaults

dependencies:

- r-base=4.3.2
- r-tidyverse
- r-sf
- r-here
- r-yaml
- r-janitor
- r-lubridate
- r-psych
- r-corrplot
- r-gparotation
- r-spdep
- r-spatstat
- r-viridis
- r-car
- r-lmtest
- r-performance
- r-broom
- r-sjplot
- r-ggplot2
- r-gridExtra
- r-knitr
- r-rmarkdown
- r-segmented
- r-poclca
- r-factoextra
- python=3.10
- pandas
- numpy
- geopandas
- matplotlib
- seaborn
- scikit-learn
- scipy
- jupyter
- pip

- pip:
- -r requirements.txt

---

## 16. requirements.txt

```
Python requirements for Riyadh PTAP Audit Framework
Generated: 2026

Core data analysis
pandas>=1.5.0
numpy>=1.24.0
scipy>=1.10.0

Spatial analysis
geopandas>=0.14.0
shapely>=2.0.0
pyproj>=3.5.0
rasterio>=1.3.0

Visualization
matplotlib>=3.7.0
seaborn>=0.12.0
plotly>=5.14.0
folium>=0.14.0

Machine learning
scikit-learn>=1.2.0

Geospatial
contextily>=1.3.0
mapclassify>=2.5.0

Jupyter
jupyter>=1.0.0
ipykernel>=6.21.0

Utilities
```

```
tqdm>=4.65.0
pyyaml>=6.0
click>=8.1.0
```

---

## 17. docker/Dockerfile

```
Dockerfile for Riyadh PTAP Audit Framework
=====
Author: Majed M Halawani
Description: Containerized environment for reproducibility
=====

Base image with R and Python
FROM rocker/geospatial:4.3.2

Install system dependencies
RUN apt-get update && apt-get install -y \
 python3-pip \
 python3-dev \
 libgdal-dev \
 libproj-dev \
 libgeos-dev \
 libudunits2-dev \
 && rm -rf /var/lib/apt/lists/*

Set working directory
WORKDIR /workspace

Copy environment files
COPY environment.yml requirements.txt ./

Install Python packages
RUN pip3 install --no-cache-dir -r requirements.txt

Install R packages
RUN Rscript -e "install.packages(c('tidyverse', 'sf', 'here', 'yaml', 'janitor', 'lubridate', 'r
```

*# Copy project files*

**COPY** ..

*# Create directories*

**RUN** mkdir -p data/raw data/processed results/figures results/tables results/su

*# Set permissions*

**RUN** chmod -R 755 /workspace

*# Default command*

**CMD** ["/bin/bash"]

---

## 18. dashboard/index.html

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
 <meta charset="UTF-8">
```

```
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
 <title>Riyadh PTAP Audit Framework - Interactive Dashboard</title>
```

```
 <link rel="stylesheet" href="style.css">
```

```
 <link href="https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;7
```

```
</head>
```

```
<body>
```

```
 <div class="dashboard">
```

```
 <!-- Header -->
```

```
 <header class="header">
```

```
 <div class="container">
```

```
 <h1>Riyadh PTAP Audit Framework</h1>
```

```
 <p class="subtitle">Climate-Adaptive Audit Framework for Transit Access
```

```
 <p class="author">Majed M Halawani | King Saud University | 2026</p>
```

```
 </div>
```

```
 </header>
```

```
 <!-- Navigation -->
```

```
 <nav class="nav">
```

```
 <div class="container">
```

```

<button class="nav-btn active" data-section="overview">Overview</button>
<button class="nav-btn" data-section="scores">Quality Scores</button>
<button class="nav-btn" data-section="spatial">Spatial Analysis</button>
<button class="nav-btn" data-section="lca">LCA Typologies</button>
<button class="nav-btn" data-section="comparison">Comparison</button>
</div>
</nav>

```

```

<!-- Dashboard Content -->

```

```

<main class="main">

```

```

 <div class="container">

```

```

 <!-- Section: Overview -->

```

```

 <section id="overview" class="section active">

```

```

 <h2>Study Overview</h2>

```

```

 <div class="stats-grid">

```

```

 <div class="stat-card">

```

```

 <div class="stat-value">60</div>

```

```

 <div class="stat-label">Total PTAPs Audited</div>

```

```

 </div>

```

```

 <div class="stat-card">

```

```

 <div class="stat-value">15</div>

```

```

 <div class="stat-label">Indicators Assessed</div>

```

```

 </div>

```

```

 <div class="stat-card">

```

```

 <div class="stat-value">0.61</div>

```

```

 <div class="stat-label">Mean Composite Score</div>

```

```

 </div>

```

```

 <div class="stat-card">

```

```

 <div class="stat-value">0.91</div>

```

```

 <div class="stat-label">Tucker's Congruence</div>

```

```

 </div>

```

```

 </div>

```

```

 <div class="info-box">

```

```

 <h3>Key Findings</h3>

```

```



```

```

 Metro vs Bus: Metro stations significantly o

```

```

 Central vs Peripheral: Central wards outper

```

```

 Spatial Clustering: Global Moran's I = 0.52 (p

```

```

 Factor Structure: Five factors explain 71.3%
 PTAP Typologies: High-Quality Integrated (

</div>
</section>

```

```

<!-- Section: Quality Scores -->
<section id="scores" class="section">
 <h2>Infrastructure Quality Scores</h2>
 <div class="chart-container">
 <div id="score-chart"></div>
 </div>
 <div class="table-container">
 <table id="score-table">
 <thead>
 <tr>
 <th>PTAP ID</th>
 <th>Type</th>
 <th>Composite Score</th>
 <th>Rank</th>
 </tr>
 </thead>
 <tbody id="score-body">
 </tbody>
 </table>
 </div>
</section>

```

```

<!-- Section: Spatial Analysis -->
<section id="spatial" class="section">
 <h2>Spatial Analysis</h2>
 <div class="spatial-grid">
 <div class="spatial-card">
 <h3>Global Moran's I</h3>
 <div class="big-number">0.52</div>
 <p>p < 0.001</p>
 <div class="interpretation">Significant positive spatial autocorrela
 </div>
 </div>

```

```

<div class="spatial-card">
 <h3>LISA Clusters</h3>

 High-High: 6
 Low-Low: 18
 Not Significant: 36

</div>
<div class="spatial-card">
 <h3>Spatial Extent</h3>

 Hot Spots: 18 km2
 Cold Spots: 72 km2

</div>
</div>
<div id="cluster-map"></div>
</section>

<!-- Section: LCA Typologies -->
<section id="lca" class="section">
 <h2>PTAP Typologies (LCA)</h2>
 <div class="lca-grid">
 <div class="lca-card class-1">
 <h3>Class 1</h3>
 <h4>High-Quality Integrated</h4>
 <div class="lca-details">
 n = 18 (30%)
 Mean = 0.78
 83% Metro
 72% Central
 </div>
 <p>Strategy: Maintain; use as benchmark</p>
 </div>
 <div class="lca-card class-2">
 <h3>Class 2</h3>
 <h4>Moderate with Lighting Deficits</h4>
 <div class="lca-details">

```

```

 n = 25 (42%)
 Mean = 0.58
 68% Bus
 56% Central
 </div>
 <p>Strategy: Targeted lighting and crossing up<
</div>
<div class="lca-card class-3">
 <h3>Class 3</h3>
 <h4>Low-Quality Basic</h4>
 <div class="lca-details">
 n = 17 (28%)
 Mean = 0.37
 88% Bus
 94% Peripheral
 </div>
 <p>Strategy: Comprehensive infrastructure inv
</div>
</div>
</section>

```

```

<!-- Section: Comparison -->
<section id="comparison" class="section">
 <h2>Comparative Assessment</h2>
 <div class="comparison-grid">
 <div class="comparison-card">
 <h3>Modified vs Original PTA</h3>

 Modified Mean: 0.61
 Original Mean: 0.56
 Difference: +0.05
 CV Modified: 32%
 CV Original: 24%
 Improvement: +33%

 </div>
 <div class="comparison-card">
 <h3>Statistical Tests</h3>

```

```


 Paired t-test: t = 4.87, p < 0.001
 Cohen's d: 0.30
 Spearman's ρ: 0.89
 Impact in peripheral areas: β = 0.18, p < 0.

 </div>
 </div>
 </section>
</div>
</main>

<!-- Footer -->
<footer class="footer">
 <div class="container">
 <p>© 2026 Majed M Halawani | King Saud University</p>
 <p>Data and code available at: GitHub Repository</p>
 </div>
</footer>
</div>

<script src="script.js"></script>
</body>
</html>

```

## 19. dashboard/style.css

```

/* =====
Dashboard Styles
Riyadh PTAP Audit Framework
===== */

* {
 margin: 0;
 padding: 0;
 box-sizing: border-box;
}

```

```

body {
 font-family: 'Inter', -apple-system, BlinkMacSystemFont, sans-serif;
 background: #f8f9fa;
 color: #1a1a2e;
 line-height: 1.6;
}

.container {
 max-width: 1200px;
 margin: 0 auto;
 padding: 0 20px;
}

/* Header */
.header {
 background: linear-gradient(135deg, #1a1a2e 0%, #16213e 50%, #0f3460 100%);
 color: white;
 padding: 40px 0;
 text-align: center;
}

.header h1 {
 font-size: 2.5rem;
 font-weight: 700;
 letter-spacing: -0.02em;
 margin-bottom: 10px;
}

.header .subtitle {
 font-size: 1.1rem;
 opacity: 0.8;
 max-width: 700px;
 margin: 0 auto;
}

.header .author {
 font-size: 0.95rem;

```

```
 opacity: 0.7;
 margin-top: 8px;
}

/* Navigation */
.nav {
 background: white;
 border-bottom: 1px solid #e9ecef;
 position: sticky;
 top: 0;
 z-index: 100;
 padding: 12px 0;
}

.nav .container {
 display: flex;
 gap: 8px;
 flex-wrap: wrap;
}

.nav-btn {
 padding: 8px 20px;
 border: none;
 background: transparent;
 border-radius: 6px;
 cursor: pointer;
 font-size: 0.9rem;
 font-weight: 500;
 color: #495057;
 transition: all 0.2s;
}

.nav-btn:hover {
 background: #f1f3f5;
 color: #1a1a2e;
}

.nav-btn.active {
```

```

 background: #0f3460;
 color: white;
}

/* Main Content */
.main {
 padding: 30px 0 60px;
}

.section {
 display: none;
 animation: fadeIn 0.3s ease;
}

.section.active {
 display: block;
}

@keyframes fadeIn {
 from { opacity: 0; transform: translateY(10px); }
 to { opacity: 1; transform: translateY(0); }
}

.section h2 {
 font-size: 1.8rem;
 font-weight: 600;
 margin-bottom: 24px;
 color: #1a1a2e;
}

/* Stats Grid */
.stats-grid {
 display: grid;
 grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
 gap: 20px;
 margin-bottom: 30px;
}

```

```
.stat-card {
 background: white;
 border-radius: 12px;
 padding: 24px;
 text-align: center;
 box-shadow: 0 2px 8px rgba(0,0,0,0.06);
 border: 1px solid #e9ecef;
}
```

```
.stat-card .stat-value {
 font-size: 2.5rem;
 font-weight: 700;
 color: #0f3460;
}
```

```
.stat-card .stat-label {
 font-size: 0.9rem;
 color: #6c757d;
 margin-top: 4px;
}
```

```
/* Info Box */
.info-box {
 background: white;
 border-radius: 12px;
 padding: 24px;
 border: 1px solid #e9ecef;
 box-shadow: 0 2px 8px rgba(0,0,0,0.06);
}
```

```
.info-box h3 {
 font-size: 1.1rem;
 font-weight: 600;
 margin-bottom: 12px;
}
```

```
.info-box ul {
 list-style: none;
```

```

padding: 0;
}

.info-box ul li {
padding: 6px 0;
border-bottom: 1px solid #f1f3f5;
}

.info-box ul li:last-child {
border-bottom: none;
}

/* Spatial Grid */
.spatial-grid {
display: grid;
grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
gap: 20px;
margin-bottom: 24px;
}

.spatial-card {
background: white;
border-radius: 12px;
padding: 20px;
border: 1px solid #e9ecef;
box-shadow: 0 2px 8px rgba(0,0,0,0.06);
}

.spatial-card h3 {
font-size: 0.95rem;
color: #495057;
margin-bottom: 12px;
}

.spatial-card .big-number {
font-size: 2.8rem;
font-weight: 700;
color: #0f3460;

```

```

}

.spatial-card ul {
 list-style: none;
 padding: 0;
}

.spatial-card ul li {
 padding: 4px 0;
 display: flex;
 justify-content: space-between;
 border-bottom: 1px solid #f1f3f5;
}

.spatial-card ul li:last-child {
 border-bottom: none;
}

.spatial-card .interpretation {
 margin-top: 8px;
 color: #6c757d;
 font-size: 0.9rem;
 font-style: italic;
}

/* LCA Grid */
.lca-grid {
 display: grid;
 grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
 gap: 24px;
 margin-top: 16px;
}

.lca-card {
 border-radius: 12px;
 padding: 24px;
 border: 1px solid #e9ecef;
 box-shadow: 0 2px 8px rgba(0,0,0,0.06);
}

```

```
}

.lca-card.class-1 {
 border-top: 4px solid #2E8B57;
}

.lca-card.class-2 {
 border-top: 4px solid #DAA520;
}

.lca-card.class-3 {
 border-top: 4px solid #CD5C5C;
}

.lca-card h3 {
 font-size: 0.85rem;
 color: #6c757d;
 text-transform: uppercase;
 letter-spacing: 0.05em;
}

.lca-card h4 {
 font-size: 1.2rem;
 margin: 4px 0 12px;
}

.lca-card .lca-details {
 display: flex;
 flex-wrap: wrap;
 gap: 8px;
 margin: 12px 0;
}

.lca-card .badge {
 background: #f1f3f5;
 padding: 4px 12px;
 border-radius: 20px;
 font-size: 0.8rem;
}
```

```

 color: #495057;
}

.lca-card p {
 font-size: 0.95rem;
 color: #495057;
}

/* Comparison Grid */
.comparison-grid {
 display: grid;
 grid-template-columns: repeat(auto-fit, minmax(300px, 1fr));
 gap: 24px;
}

.comparison-card {
 background: white;
 border-radius: 12px;
 padding: 24px;
 border: 1px solid #e9ecef;
 box-shadow: 0 2px 8px rgba(0,0,0,0.06);
}

.comparison-card h3 {
 font-size: 1.1rem;
 font-weight: 600;
 margin-bottom: 16px;
 color: #1a1a2e;
}

.comparison-card ul {
 list-style: none;
 padding: 0;
}

.comparison-card ul li {
 padding: 6px 0;
 border-bottom: 1px solid #f1f3f5;
}

```

```
display: flex;
justify-content: space-between;
}
```

```
.comparison-card ul li:last-child {
 border-bottom: none;
}
```

```
.comparison-card ul li strong {
 color: #0f3460;
}
```

```
/* Table */
```

```
.table-container {
 background: white;
 border-radius: 12px;
 padding: 20px;
 border: 1px solid #e9ecef;
 overflow-x: auto;
 margin-top: 20px;
}
```

```
.table-container table {
 width: 100%;
 border-collapse: collapse;
 font-size: 0.9rem;
}
```

```
.table-container th {
 background: #f1f3f5;
 padding: 10px 12px;
 text-align: left;
 font-weight: 600;
 color: #495057;
}
```

```
.table-container td {
 padding: 8px 12px;
```

```

border-bottom: 1px solid #f1f3f5;
}

.table-container tr:hover td {
 background: #f8f9fa;
}

.table-container .rank-high {
 color: #2E8B57;
 font-weight: 600;
}

.table-container .rank-medium {
 color: #DAA520;
}

.table-container .rank-low {
 color: #CD5C5C;
}

/* Footer */
.footer {
 background: #1a1a2e;
 color: rgba(255,255,255,0.7);
 padding: 24px 0;
 text-align: center;
 font-size: 0.9rem;
 border-top: 1px solid #2d2d4a;
}

.footer a {
 color: #4a9eff;
 text-decoration: none;
}

.footer a:hover {
 text-decoration: underline;
}

```

```
/* Responsive */
@media (max-width: 768px) {
 .header h1 {
 font-size: 1.8rem;
 }

 .header .subtitle {
 font-size: 1rem;
 }

 .nav .container {
 justify-content: center;
 }

 .nav-btn {
 padding: 6px 14px;
 font-size: 0.8rem;
 }

 .stats-grid {
 grid-template-columns: repeat(2, 1fr);
 }

 .spatial-grid {
 grid-template-columns: 1fr;
 }

 .lca-grid {
 grid-template-columns: 1fr;
 }

 .comparison-grid {
 grid-template-columns: 1fr;
 }
}

@media print {
```

```

.nav {
 display: none;
}

.section {
 display: block !important;
 page-break-inside: avoid;
}
}

```

---

## 20. dashboard/script.js

```

// =====
// Dashboard Script
// Riyadh PTAP Audit Framework
// =====

document.addEventListener('DOMContentLoaded', function() {
 // =====
 // Navigation
 // =====

 const navButtons = document.querySelectorAll('.nav-btn');
 const sections = document.querySelectorAll('.section');

 navButtons.forEach(button => {
 button.addEventListener('click', function() {
 // Update active button
 navButtons.forEach(btn => btn.classList.remove('active'));
 this.classList.add('active');

 // Show corresponding section
 const sectionId = this.dataset.section;
 sections.forEach(section => {
 section.classList.remove('active');
 if (section.id === sectionId) {
 section.classList.add('active');

```

```

 }
 });
});
});

// =====
// Sample Data
// =====

const sampleData = [
 { id: 'M001', type: 'Metro', score: 0.89, rank: 1 },
 { id: 'M002', type: 'Metro', score: 0.85, rank: 2 },
 { id: 'M003', type: 'Metro', score: 0.82, rank: 3 },
 { id: 'M004', type: 'Metro', score: 0.78, rank: 4 },
 { id: 'M005', type: 'Metro', score: 0.76, rank: 5 },
 { id: 'B001', type: 'Bus', score: 0.72, rank: 6 },
 { id: 'B002', type: 'Bus', score: 0.68, rank: 7 },
 { id: 'B003', type: 'Bus', score: 0.65, rank: 8 },
 { id: 'B004', type: 'Bus', score: 0.61, rank: 9 },
 { id: 'B005', type: 'Bus', score: 0.58, rank: 10 },
 { id: 'B006', type: 'Bus', score: 0.54, rank: 11 },
 { id: 'B007', type: 'Bus', score: 0.49, rank: 12 },
 { id: 'B008', type: 'Bus', score: 0.45, rank: 13 },
 { id: 'B009', type: 'Bus', score: 0.41, rank: 14 },
 { id: 'B010', type: 'Bus', score: 0.37, rank: 15 },
];

// =====
// Score Chart (Simple Bar Chart using CSS/HTML)
// =====

function renderScoreChart() {
 const container = document.getElementById('score-chart');
 if (!container) return;

 let html = '<div class="chart-bars">';

 const sorted = [...sampleData].sort((a, b) => b.score - a.score);

```

```

sorted.forEach(item => {
 const pct = item.score * 100;
 const color = item.type === 'Metro' ? '#2E8B57' : '#CD5C5C';
 html += `
 <div class="chart-bar-container">
 <div class="chart-label">${item.id}</div>
 <div class="chart-bar-track">
 <div class="chart-bar" style="width: ${pct}%; background: ${color};">
 ${item.score.toFixed(2)}
 </div>
 </div>
 <div class="chart-type">${item.type}</div>
 </div>
 `;
});

html += '</div>';

// Add styles for chart
const style = document.createElement('style');
style.textContent = `
.chart-bars {
 display: flex;
 flex-direction: column;
 gap: 6px;
}
.chart-bar-container {
 display: flex;
 align-items: center;
 gap: 10px;
 font-size: 0.85rem;
}
.chart-label {
 min-width: 50px;
 font-weight: 500;
 color: #495057;
}

```

```

.chart-bar-track {
 flex: 1;
 height: 28px;
 background: #f1f3f5;
 border-radius: 14px;
 overflow: hidden;
 position: relative;
}
.chart-bar {
 height: 100%;
 border-radius: 14px;
 display: flex;
 align-items: center;
 justify-content: flex-end;
 padding-right: 8px;
 transition: width 0.8s ease;
 min-width: 30px;
}
.chart-value {
 color: white;
 font-size: 0.75rem;
 font-weight: 600;
 text-shadow: 0 1px 2px rgba(0,0,0,0.3);
}
.chart-type {
 min-width: 45px;
 font-size: 0.75rem;
 color: #6c757d;
}
`;
document.head.appendChild(style);

container.innerHTML = html;
}

renderScoreChart();

```

```
// =====
```

```

// Score Table
// =====

function renderScoreTable() {
 const tbody = document.getElementById('score-body');
 if (!tbody) return;

 const sorted = [...sampleData].sort((a, b) => b.score - a.score);

 let html = '';
 sorted.forEach(item => {
 const rankClass = item.rank <= 5 ? 'rank-high' :
 item.rank <= 10 ? 'rank-medium' : 'rank-low';
 html += `
 <tr>
 <td>${item.id}</td>
 <td>${item.type}</td>
 <td>${item.score.toFixed(3)}</td>
 <td class="${rankClass}">#${item.rank}</td>
 </tr>
 `;
 });

 tbody.innerHTML = html;
}

renderScoreTable();

// =====
// LCA Typology Cards
// =====

// Data is already in the HTML

// =====
// Spatial Statistics (Update from HTML)
// =====

```

```

// The values are already in the HTML

// =====
// Responsive Chart Rendering
// =====

function handleResize() {
 // Charts will reflow naturally
}

window.addEventListener('resize', handleResize);

// =====
// Print Support
// =====

const stylePrint = document.createElement('style');
stylePrint.textContent = `
 @media print {
 .nav { display: none; }
 .section { display: block !important; page-break-inside: avoid; }
 .stat-card, .spatial-card, .lca-card, .comparison-card, .info-box, .table-cont
 box-shadow: none !important;
 border: 1px solid #ddd !important;
 }
 .header { background: #1a1a2e !important; -webkit-print-color-adjust: exac
 .chart-bar { -webkit-print-color-adjust: exact !important; }
 }
`;
document.head.appendChild(stylePrint);

console.log('Riyadh PTAP Audit Framework Dashboard loaded successfully.');
```

## 21. makefile

```
=====
Makefile for Riyadh PTAP Audit Framework
=====
Author: Majed M Halawani
Description: Automation for running all analyses
=====
```

**.PHONY:** help setup clean data analyze figures dashboard docker all

*# Default target*

**help:**

```
@echo "Riyadh PTAP Audit Framework - Makefile Commands:"
@echo ""
@echo " make setup - Install required packages"
@echo " make data - Prepare and clean data"
@echo " make analyze - Run all analyses"
@echo " make figures - Generate figures and tables"
@echo " make dashboard - Build interactive dashboard"
@echo " make docker - Build Docker image"
@echo " make all - Run full workflow"
@echo " make clean - Remove generated files"
```

*# Setup environment*

**setup:**

```
@echo "Setting up environment..."
conda env create -f environment.yml || pip install -r requirements.txt
@echo "Setup complete."
```

*# Data preparation*

**data:**

```
@echo "Preparing data..."
Rscript scripts/01_data_preparation.R
@echo "Data preparation complete."
```

*# Run all analyses*

**analyze: data**

```
@echo "Running all analyses..."
Rscript scripts/02_descriptive_statistics.R
Rscript scripts/03_pca_factor_analysis.R
Rscript scripts/04_spatial_analysis.R
Rscript scripts/05_regression_analysis.R
Rscript scripts/06_latent_class_analysis.R
Rscript scripts/07_comparative_assessment.R
Rscript scripts/08_sensitivity_analysis.R
@echo "All analyses complete."
```

*# Generate figures and tables*

**figures: analyze**

```
@echo "Generating figures and tables..."
Rscript scripts/09_figures_tables.R
@echo "Figures and tables complete."
```

*# Build dashboard*

**dashboard:**

```
@echo "Building dashboard..."
@echo "Dashboard files are in dashboard/ directory"
@echo "Open dashboard/index.html in a web browser"
```

*# Build Docker image*

**docker:**

```
@echo "Building Docker image..."
docker build -t riyadh-ptap -f docker/Dockerfile .
@echo "Docker image built: riyadh-ptap"
```

*# Run full workflow*

**all: setup data analyze figures dashboard**

```
@echo "Full workflow complete."
```

*# Clean generated files*

**clean:**

```
@echo "Cleaning generated files..."
rm -rf data/processed/*
rm -rf results/figures/*.png
rm -rf results/tables/*.csv
```

```
rm -rf results/supplementary/*.csv
rm -rf results/supplementary/*.png
@echo "Clean complete."
```

---